Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea practică Nr. 5

Disciplina: Tehnici de programare

Tema: „Funcții, metode și importuri pe web (Python)”

A efectuat : st. gr.SI – 263 Boicu Ecaterina

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

1. Importarea modulelor din biblioteca standard și externe (import, from … import, alias) și efectuarea cererilor web de bază – exercițiile 1–8;
2. Utilizarea metodelor și atributelor obiectelor web: răspuns HTTP, antete, șiruri de caractere, cookie-uri, sesiuni – exercițiile 9–20;
3. Scrierea funcțiilor proprii: parametri, valori implicite, docstring-uri, adnotări de tip, tratarea erorilor – exercițiile 21–34;
4. Utilizarea altor module din biblioteca standard (urllib.parse, re, html.parser, hashlib, json, socket, ssl) – exercițiile 35–43;
5. Crearea modulului propriu webtools.py, a programului main.py și a raportului site-ului în formate CSV și JSON – exercițiile 44–50.

Pregătirea mediului de lucru

Cerință: Verificați versiunea Python (python --version, 3.9 sau mai nouă), instalați biblioteca requests (pip install requests) și creați directorul web_lab, în care veți salva fișierele part1_imports.py, part2_methods.py etc., fiecare începând cu șablonul din lucrare (BASE_URL, ECHO_URL, TIMEOUT).

[În Visual Studio Code , am creat directorul web_lab , în care am salvat fișierele de lucru , incepand de la : part1_imports.py , pana la ultima parte.]



Fig.1- Instalarea bibliotecii requests (pip install request) în directorul web_lab

Partea 1. Importuri

Fișier de lucru: part1_imports.py (exercițiile 1–8).

Exercițiul 1. Primul import

Cerință: Importați requests și afișați versiunea acestuia (requests.__version__); apoi importați urllib.request. Explicați într-un comentariu de ce requests a necesitat pip install, iar urllib nu.

[Am importat biblioteca requests cu import requests și modulul standard urllib.requests , apoi am afisat versiunea cu requests.__version__. Am rulat fisierul part1_imports.py in terminalul VS Code , iar rezultatul a fost : Versiunea requests : 2.32.5. Avertismentul - NotOpenSSLWarning nu este o eroare, ci apare din cauza LibreSSL de pe macOS]

[requests este o biblioteca externă , care nu vine cu Python , deci trebuie instalată cu pip install requests.urllib face parte din biblioteca standard , deci este inclus in Python și se importă direct , fara instalare.]

import requests

import urllib.request

print("Versiunea requests:", requests.__version__)

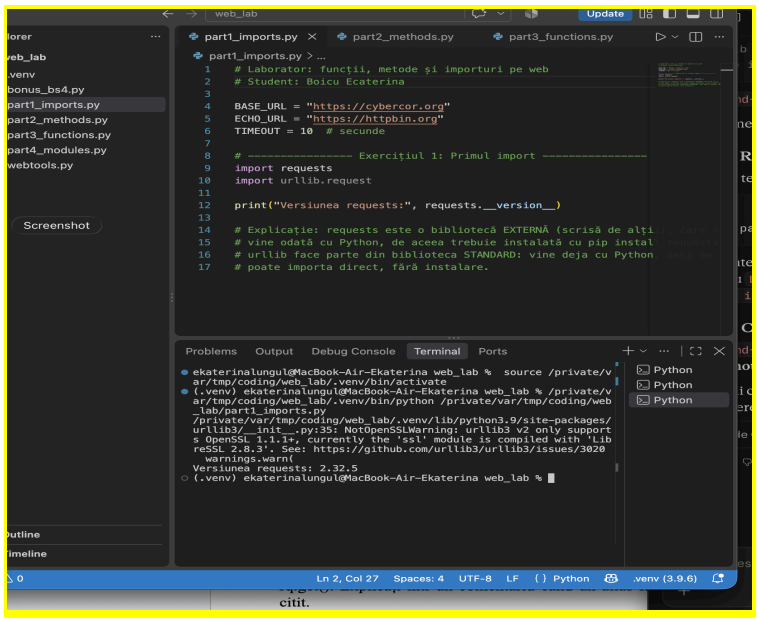


Fig.1.1- Importul requests

Exercițiul 2. Două stiluri de import

Cerință: Descărcați pagina BASE_URL cu requests.get() și afișați codul de stare; rescrieți scriptul cu from requests import get. Scrieți în comentarii câte un avantaj al fiecărui stil.

[Am descarcat pagina BASE_URL în doua moduri: cu import requests și requests.get() , apoi cu from requests import get și get () , cu pauza time.sleep(1) între cereri . Ambele variante au returnat codul de stare 200 , deci site-ul răspunde corect.]

[Avantajul import requests este ca se vede clar de unde vine funcția (requests.get) și nu apar conflicte de nume. Avantajul from requests import get este ca codul e mai scurt , fără prefixul requests.]

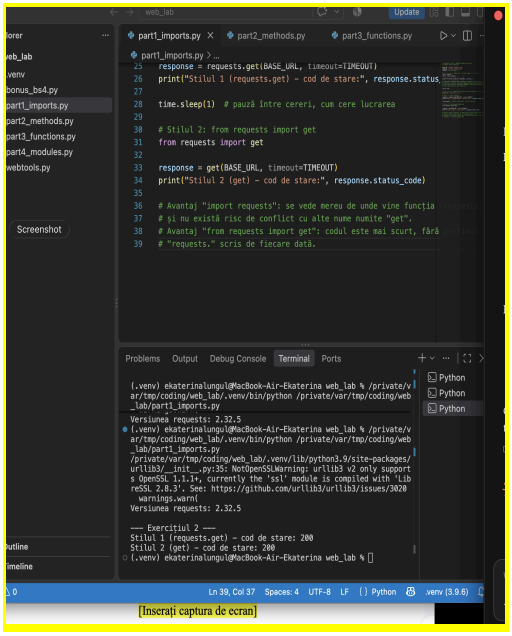


Fig.1.2- Două moduri de import

Exercițiul 3. Alias-uri

Cerință: Importați requests cu alias (import requests as rq) și descărcați pagina principală cu rq.get(). Explicați într-un comentariu când un alias face codul mai ușor și când mai greu de citit.

[Am importat requests cu alias (import requests as rq) și am descarcat pagina principala cu rq.get () . Rezultatul a fost codul de stare 200.]

[Un alias face codul mai usor de citit cand numele modulului este lung sau cand alias-ul este consacrat (de exemplu , np pentru numpy). Îl face mai greu de citit cand alias-ul este scurt si neclar (rq,r,x) , pentru ca cititorul trebuie sa ghicească ce înseamnă.]

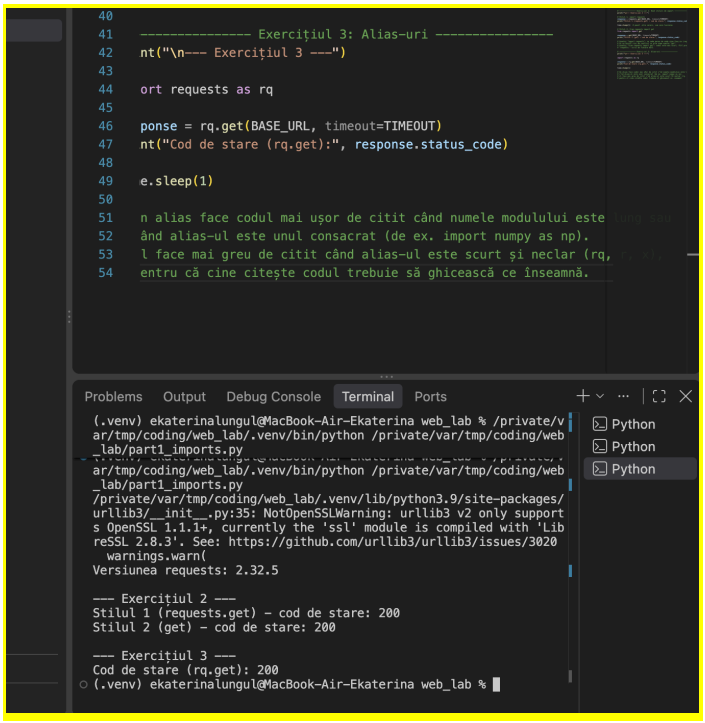


Fig.1.3 Importul requests cu alias

Exercițiul 4. Doar biblioteca standard

Cerință: Descărcați pagina principală folosind doar urllib.request.urlopen(); afișați atributul .status și primele 200 de caractere din corp (octeții se decodifică în șir de caractere).

[Am descarcat pagina principala doar cu urllib.request.urlopen() , am aflat atributul .status și primele 200 de caractere din corp , decodificat din octeți cu .decode("utf-8") .La prima încercare serverul a răspuns 403 Forbidden , deoarece blochează User-Agent-ul implicit Python-urllib. Am rezolvat trimitand un obiect Request cu un User-Agent propriu , iar rezultatul a fost Status: 200.]

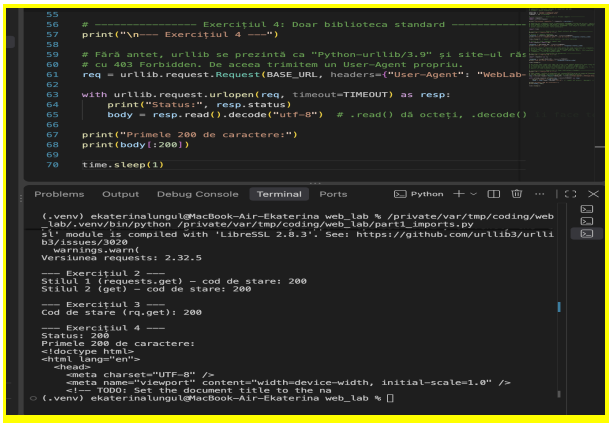


Fig.1.4 Biblioteca standard

Exercițiul 5. Priviți în interiorul unui modul

Cerință: Rulați print(dir(requests)); alegeți trei nume din rezultat și indicați într-un comentariu dacă fiecare este funcție, clasă sau modul.

[Am afișat conținutul modulului cu print(dir(requests)) și am ales trei nume , al căror tip l-am verificat cu modulul inspect.]

[requests.get este o funcție , requests.Session este o clasa , iar requests.exceptions este un modul.]

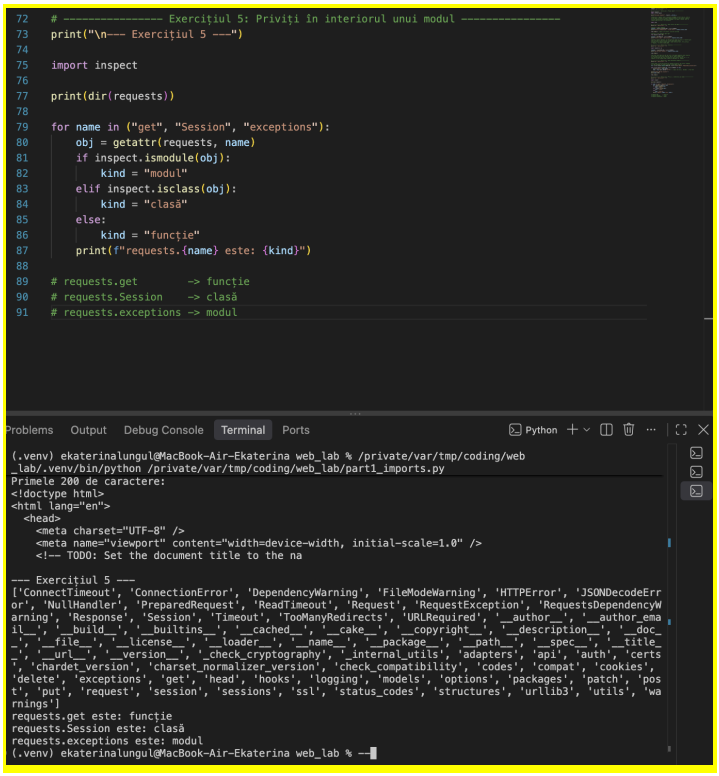


Fig.1.5- Interiorul unui modul

Exercițiul 6. Citiți documentația

Cerință: Rulați help(requests.get), găsiți parametrul care setează timpul maxim de așteptare și folosiți-l într-o cerere.

[Am rulat help(request.get) și am citit documentația funcției . Parametrul care seteaza timpul maxim de așteptare este timeout(în secunde) , descris în documentația request.request, la care trimite **kwargs. L-am folosit intr-o cerere cu timeout=TIMEOUT , care a returnat codul de stare 200.]

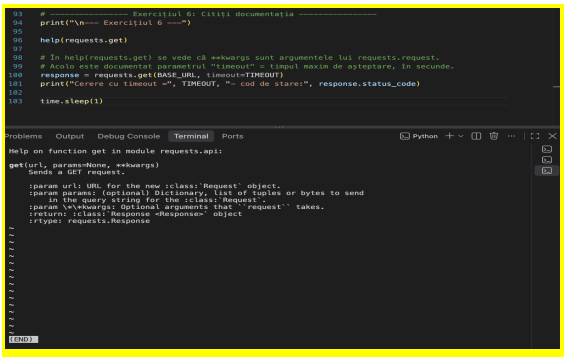


Fig.1.6 - help(requests.get),programul s-a oprit la (END) și așteaptă tastarea “q”

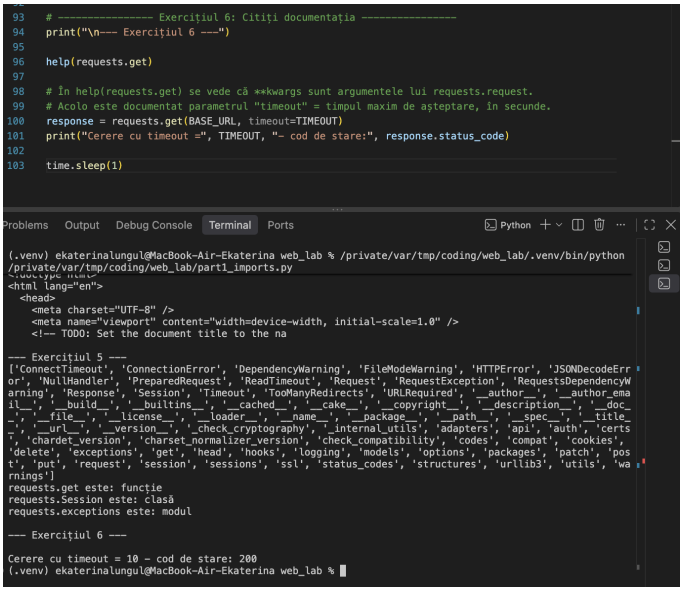


Fig.1.6.1- Citirea documentației finale

Exercițiul 7. Cronometrarea unei cereri

Cerință: Importați time și măsurați durata unei cereri către pagina principală cu time.perf_counter() (înainte și după); comparați rezultatul cu response.elapsed.

[Am importat time și am măsurat durata unei cereri de către pagina principala cu time.perf_counter () ,apelat înainte și după cerere. Am comparat rezultatul cu response.elapsed.]

[Valoarea din perf_counter() este de obicei puțin mai mare , pentru ca măsoară tot timpul din program , inclusiv citirea corpului și munca internă a bibliotecii.response.elapsed măsoară doar intervalul dintre trimiterea cererii și primirea antetelor răspunsului.]

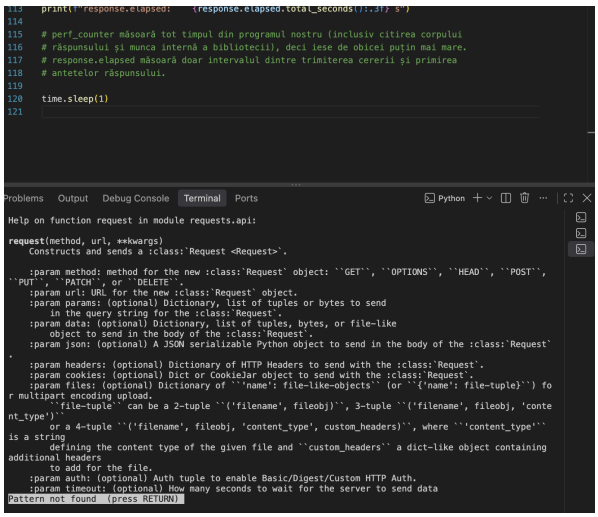


Fig.1.7 -help(requests.get),programul s-a oprit la (END) și așteaptă tastarea “q”

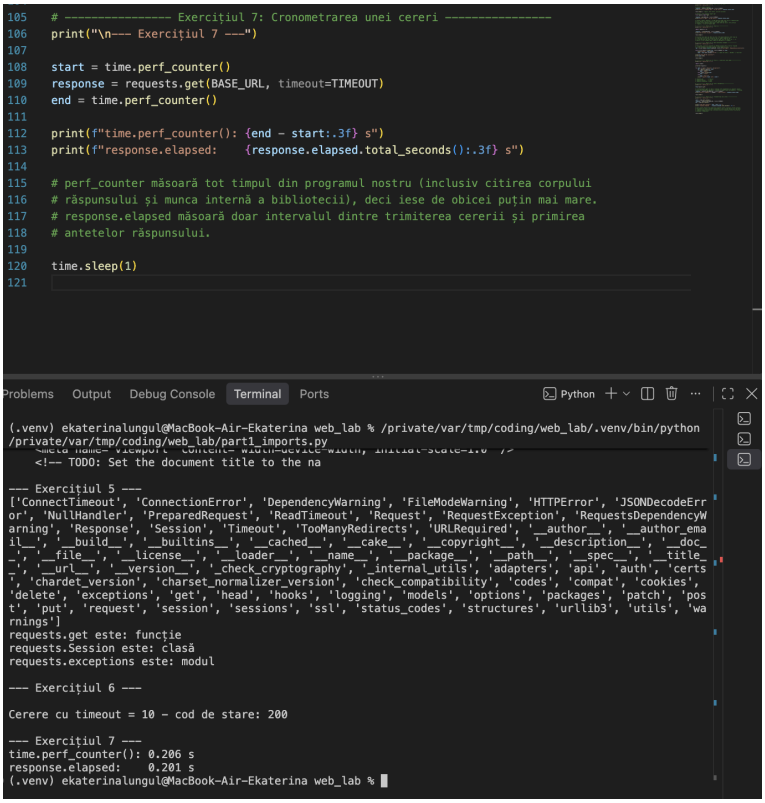


Fig.1.7.1- Cronometrarea cererii

Exercițiul 8. Când un import eșuează

Cerință: Scrieți un script care încearcă import bs4 într-un bloc try / except ImportError; dacă modulul lipsește, afișați mesajul de instalare cerut în lucrare.

[Am scris un script care încearcă import bs4 intr-un bloc try / except ImportError.Deoarece modulul nu era instalat, programul nu s-a oprit cu eroare , ci a afișat mesajul -Instalați modulul cu : pip install beautifulsoup4.]

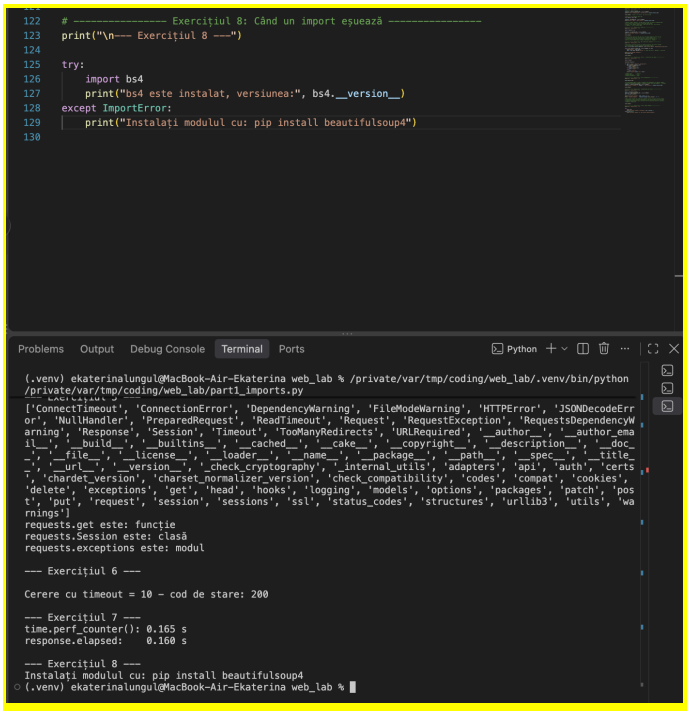


Fig.1.8- Încercarea importului bs4

Întrebare de verificare

Întrebare: Care este diferența dintre un modul, un pachet și o bibliotecă?

[Un modul este singur fișier Python(.py) care contine cod reutilizabil.Un pachet este un director de module , uneori cu subpachete (de exemplu - urllib). O biblioteca este o colectie de module sau pachete cu un scop comun (de exemplu - requests).]

Partea 2. Metode și atribute

Fișier de lucru: part2_methods.py (exercițiile 9–20).

Exercițiul 9. Elementele de bază ale răspunsului

Cerință: Descărcați pagina principală și afișați response.status_code, response.ok, response.url și response.encoding; lângă fiecare linie comentați dacă este atribut sau metodă.

[Am descarcat pagina principala și am afisat response.status_code,response.ok,response.url și response.encoding. Toate sunt atribute (citite fără paranteze) , iar rezultatul a fost : cod 200,ok=True, URL-ul paginii și codificarea utf-8.]

[status_code,ok,url și encoding sunt atribute , deoarece sunt valori citite direct din obiect , fără paranteze.]

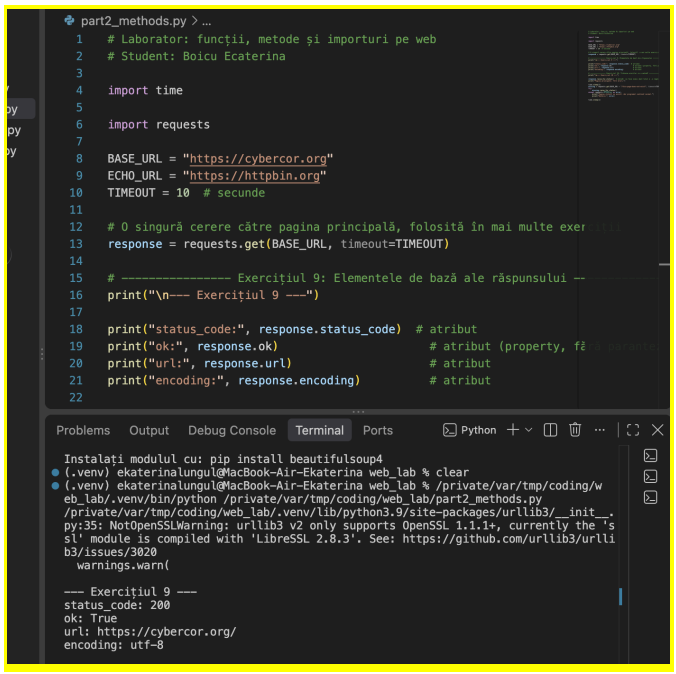


Fig.2.9 - Inserarea metodelor și atributelor

Exercițiul 10. Tratarea erorilor cu o metodă

Cerință: Apelați response.raise_for_status() pe pagina principală; apoi trimiteți o cerere către BASE_URL + "/this-page-does-not-exist", prindeți requests.HTTPError și afișați un mesaj prietenos în loc de oprirea programului.

[Am apelat metode response.raise_for_status() pe pagina principala , care nu a produs eroare. Apoi am cerut o pagina inexistenta si am prins excepția requests.HTTPError cu try / except , afisand un mesaj prietenos în loc de oprirea programului.]

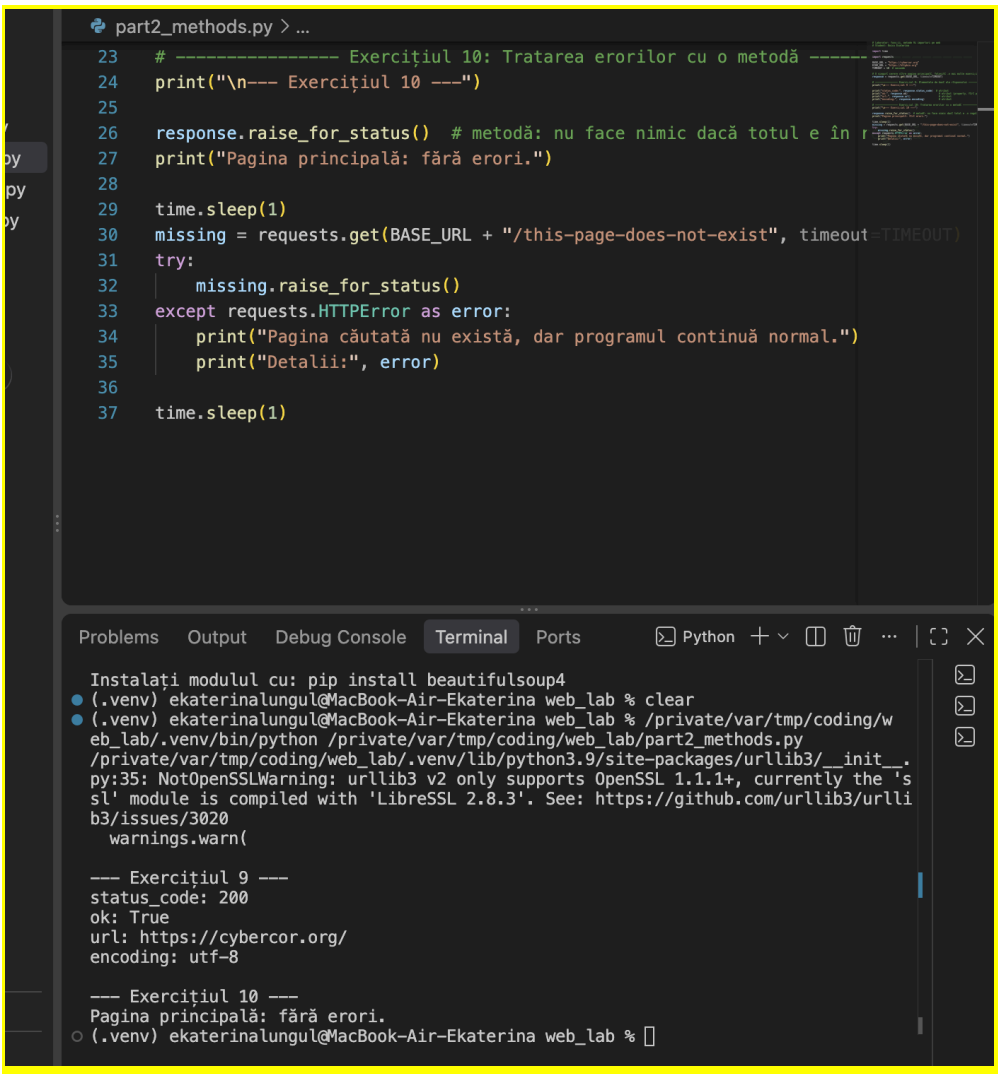


Fig.2.10- Apelarea metodei response.raise_for_status()

Exercițiul 11. Toate antetele

Cerință: Parcurgeți response.headers.items() și afișați fiecare antet în forma Nume: valoare.

[Am parcurs response.headers.items() într-o bucla for și am afisat fiecare antet în forma Nume : valoare .]

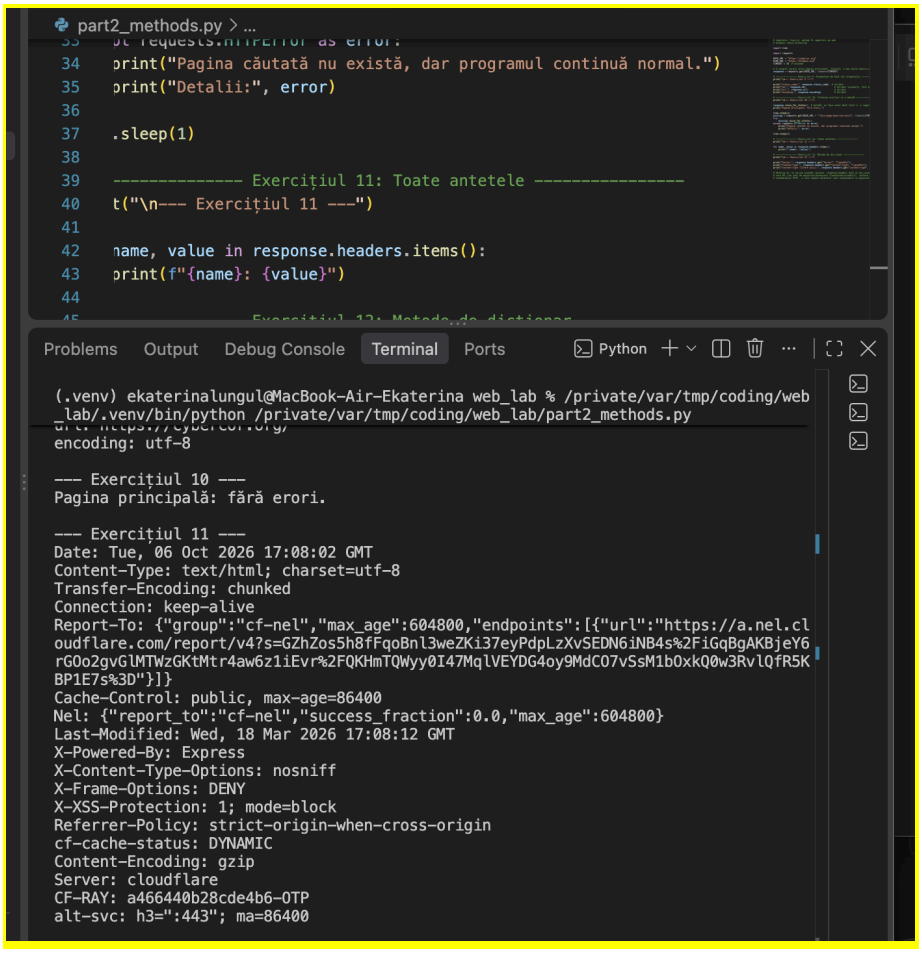


Fig.2.11.- Afisarea antetelor

Exercițiul 12. Metode de dicționar

Cerință: Cu response.headers.get() afișați antetele Server și Content-Type (valoare implicită „lipsește”); încercați și varianta „content-type”, cu litere mici, și notați ce observați.

[Am afisat antetele Server și Content - Type cu response.headers.get () , folosind “lipsește” ca valoare implicită. Apoi am încercat și varianta content -type ,cu litere mici.]

[Varianta cu litere mici returnează aceeași valoare, deoarece response.headers este un dicționar insensibil la majuscule/minuscule ,conform standardului HTTP..]

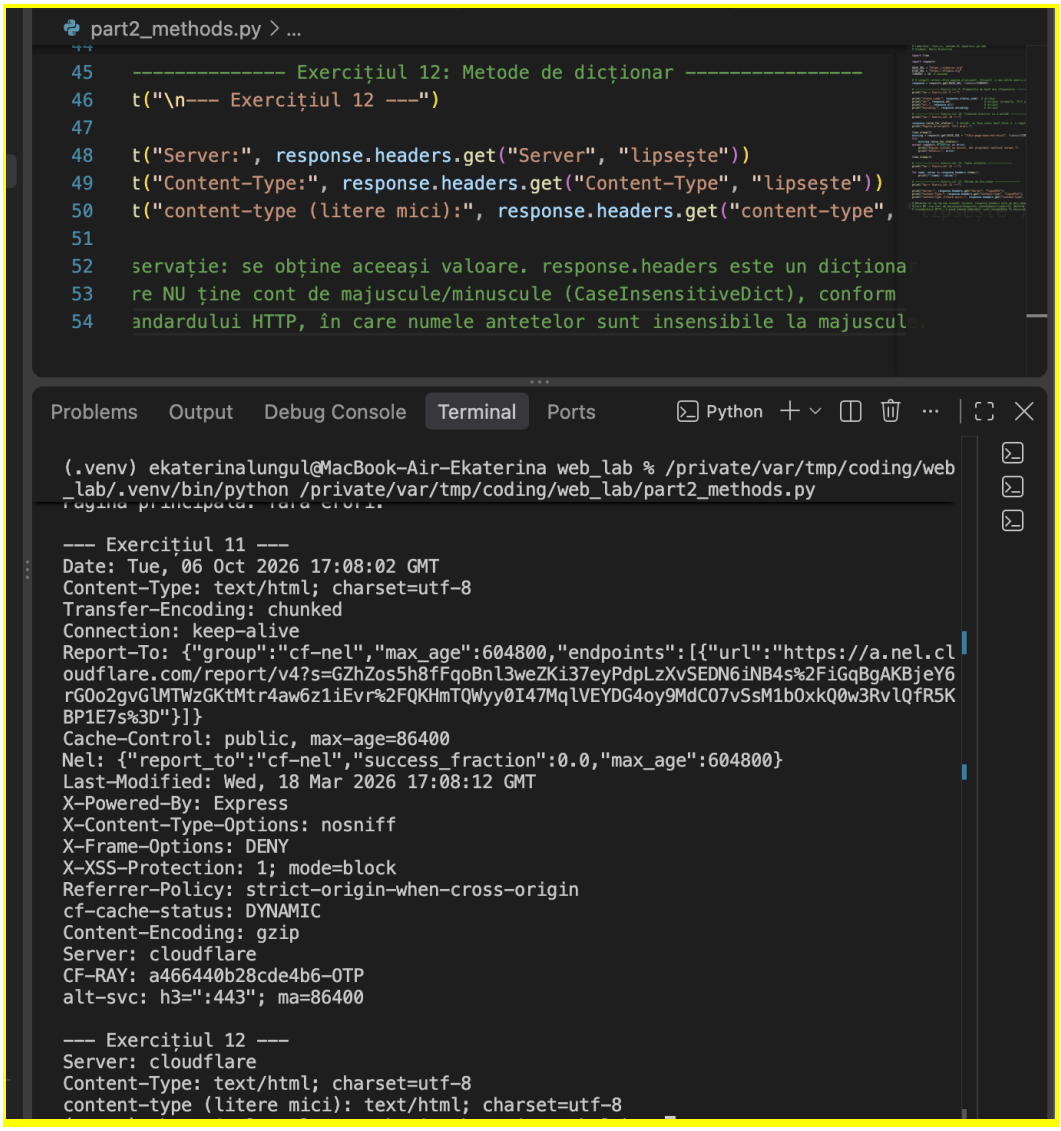


Fig.2.12.- Citirea dicționarelor

Exercițiul 13. Înlănțuirea metodelor de șir

Cerință: Numărați de câte ori apare „cyber” pe pagina principală cu response.text.lower().count("cyber"); explicați într-un comentariu de ce se pot înlănțui .lower() și .count().

[Am numarat de cate ori apare cuvântul “ cyber” pe pagina principala cu response.text.lower().count(“cyber”).]

[Metodele se pot inlantui, deoarece .lower() returnează un șir nou , iar pe acesta se poate apela direct .count().]

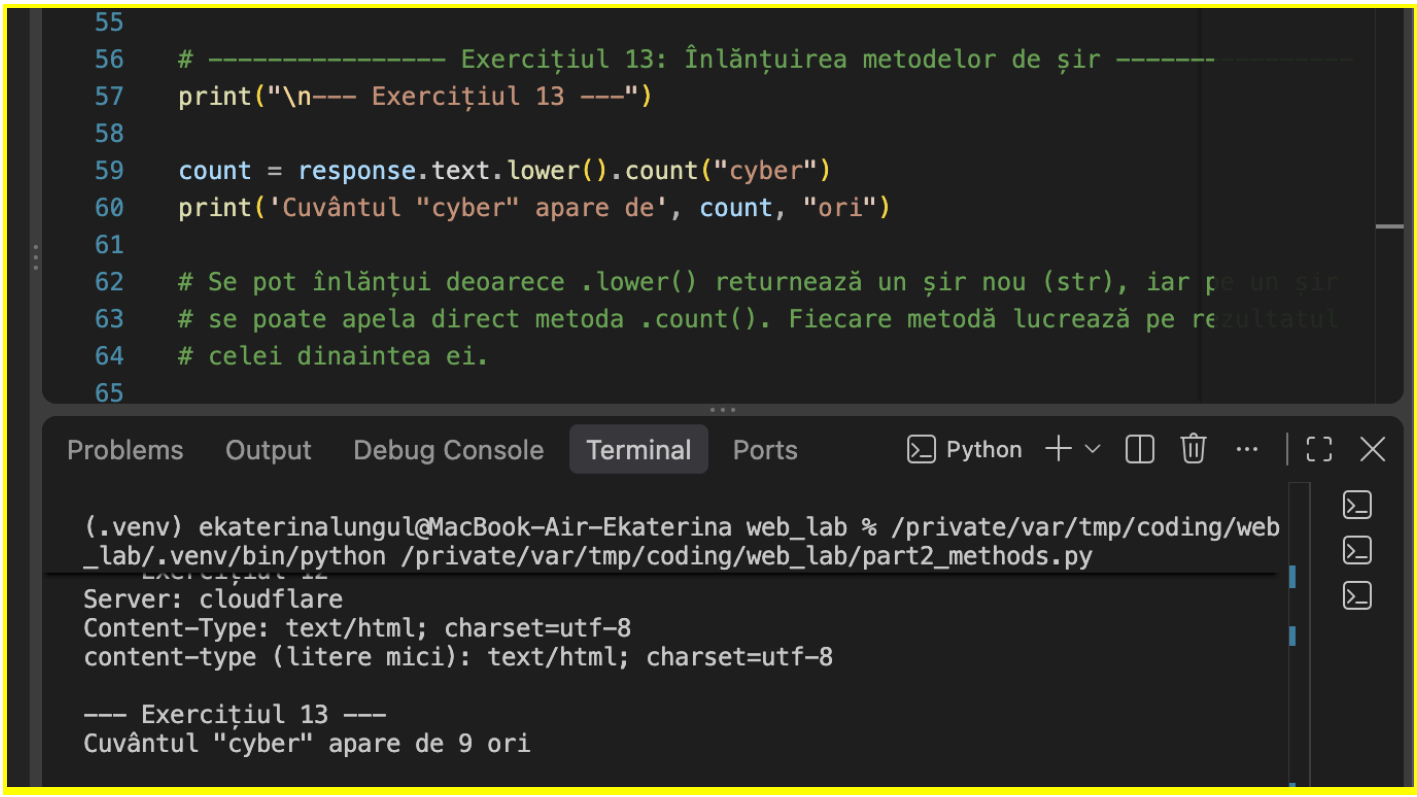


Fig.2.13- Afișarea rezultatului frecvenței cuvantului “cyber”

Exercițiul 14. Găsirea titlului

Cerință: Extrageți textul dintre <title> și </title> folosind .find() și felierea (slicing) pe response.text; curățați-l cu .strip() și afișați-l.

[Am extras titlul paginii folosind .find() pentru poziția dintre <title> si </title> , apoi secționarea (slicing) și .strip() pentru curatare.]

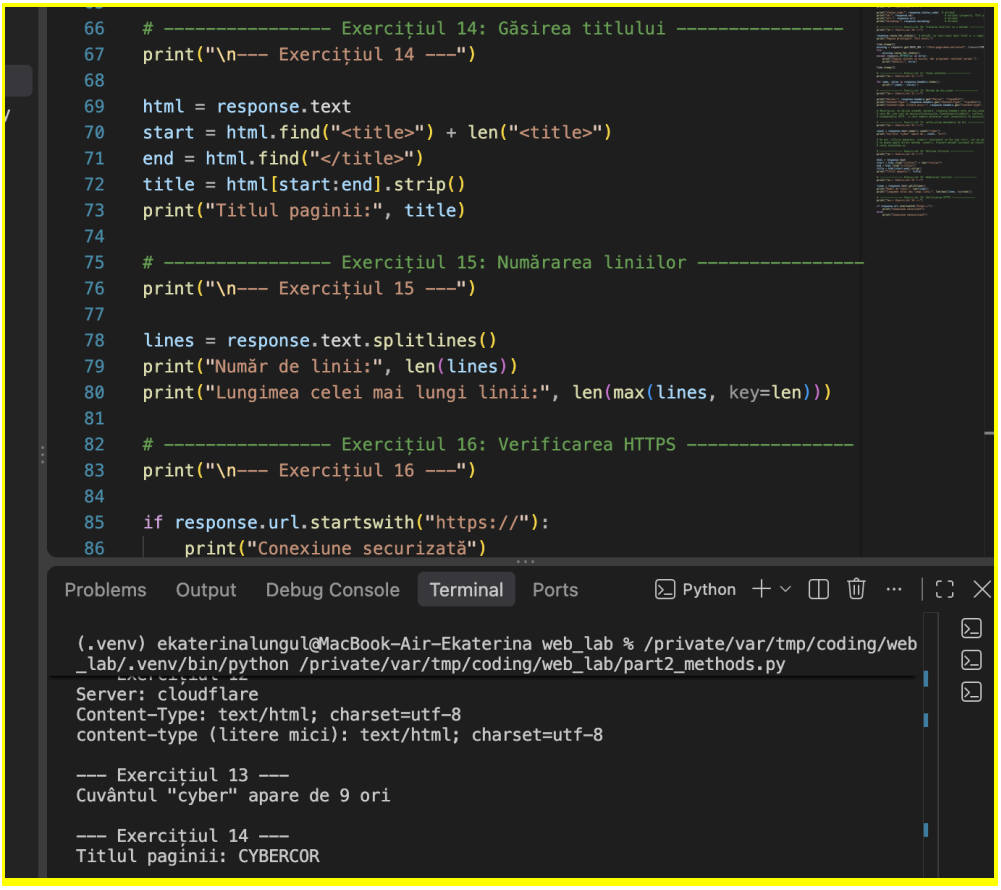


Fig.2.14.- Afișarea titlului

Exercițiul 15. Numărarea liniilor

Cerință: Împărțiți codul HTML în linii cu .splitlines(); afișați numărul de linii și lungimea celei mai lungi linii (max(lines, key=len)).

[Am împărțit codul HTML în linii cu .splitlines () și am afișat numărul de linii și lungimea celei mai lungi linii , calculata cu max(lines, key=len).]

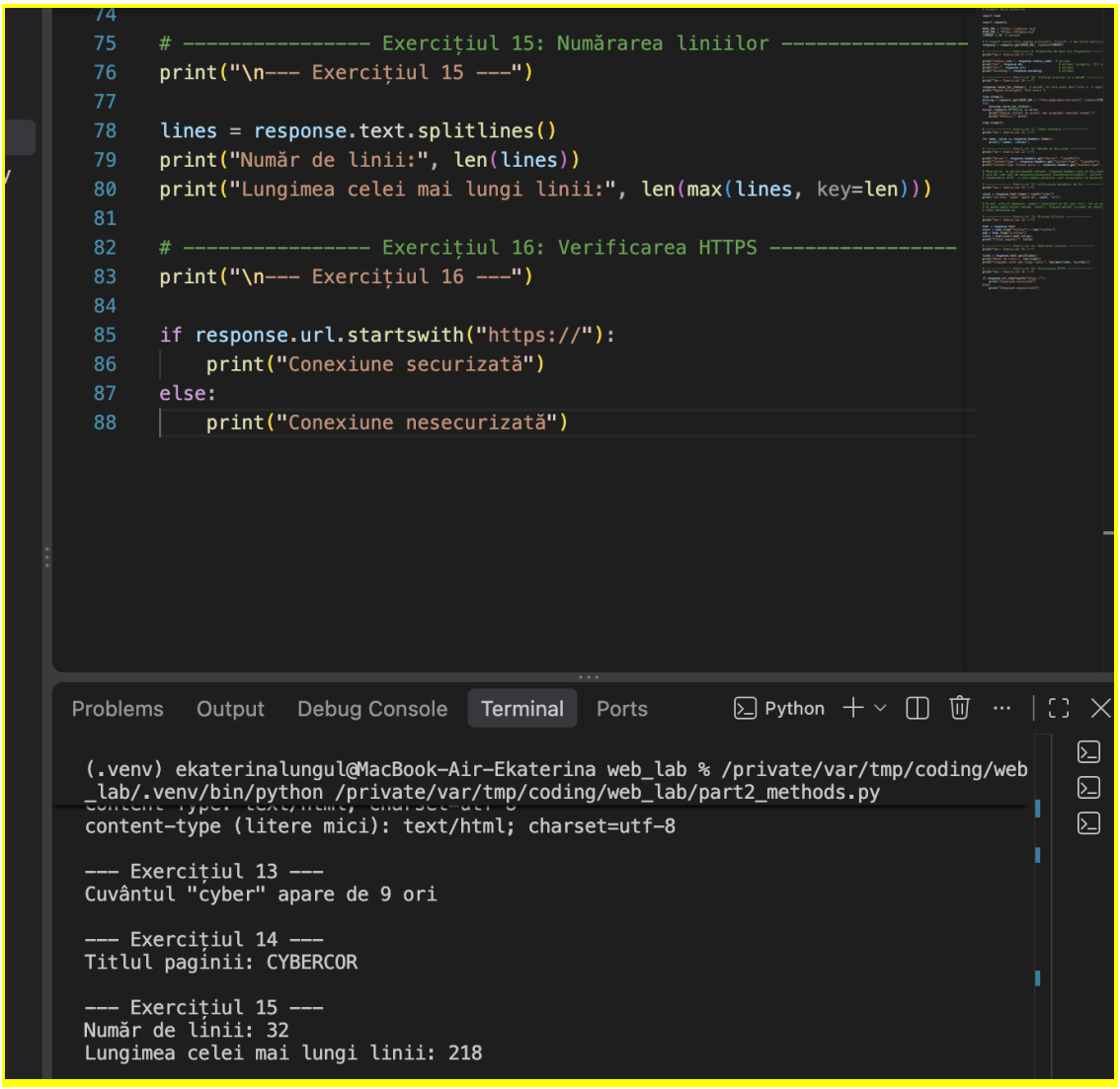


Fig.2.15.- Executarea itemul 15

Exercițiul 16. Verificarea HTTPS

Cerință: Folosiți .startswith() pentru a verifica dacă response.url începe cu https:// și afișați „Conexiune securizată” sau „Conexiune nesecurizată”.

[Am verificat cu .startswith(“https://”) dacă response.url începe cu https://.

Rezultatul“ Conexiune securizata”.]

```python
# ---------- Exercițiul 16: Verificarea HTTPS ----------
print("\n--- Exercițiul 16 ---")

if response.url.startswith("https://"):
    print("Conexiune securizată")
else:
    print("Conexiune nesecurizată")
```

```
Problems  Output  Debug Console  Terminal  Ports  Python

(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab % /private/var/tmp/coding/web
_lab/.venv/bin/python /private/var/tmp/coding/web_lab/part2_methods.py
content-type (litere mici): text/html; charset=utf-8

--- Exercițiul 13 ---
Cuvântul "cyber" apare de 9 ori

--- Exercițiul 14 ---
Titlul paginii: CYBERCOR

--- Exercițiul 15 ---
Număr de linii: 32
Lungimea celei mai lungi linii: 218

--- Exercițiul 16 ---
Conexiune securizată
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab %
```

Fig.2.16.- Verificarea conexiunii

Exercițiul 17. Urmărirea redirecționărilor

Cerință: Trimiteți o cerere către http://cybercor.org (fără S); parcurgeți response.history, afișând codul de stare și URL-ul fiecărei redirecționări, apoi URL-ul final.

[Am trimis o cerere catre http://cybercor.org (fara S) și am parcurs response.history , afisand codul de stare și URL-ul fiecărei redirectionari , apoi URL-ul final.]

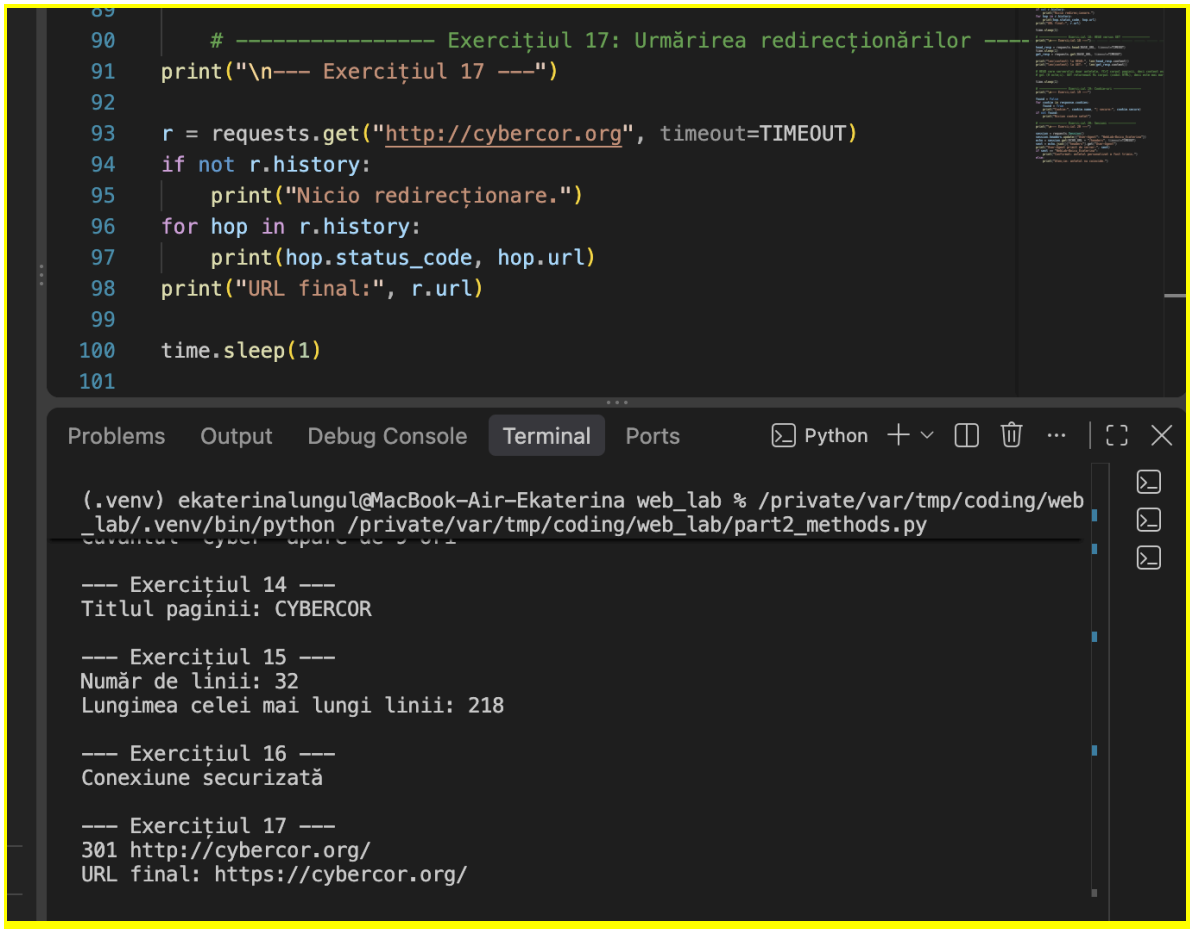


Fig.2.17. Afișarea cererii

Exercițiul 18. HEAD versus GET

Cerință: Trimiteți către pagina principală o cerere requests.head() și una requests.get(); comparați len(response.content) și explicați diferența într-un comentariu.

[Am trimis către pagina principala o cerere requests.head () și una request.get()și am comparat len(response.content) ambele.]

[HEAD returnează doar antetele , fără corpul paginii , deci content are 0 octeți.GET returnează și corpul (HTML) , deci conținutul este mai mare .]

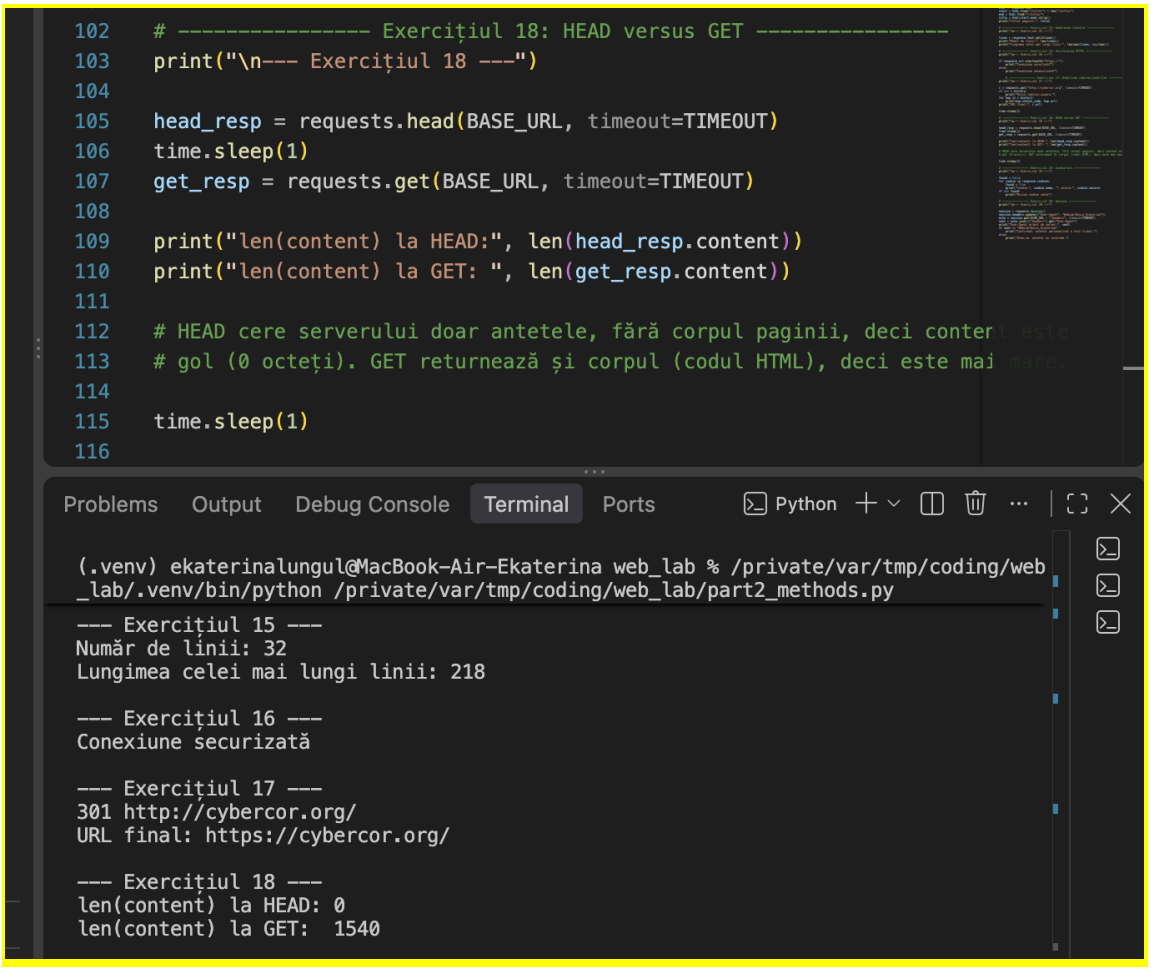


Fig.2.18.- Rezultatul comparării

Exercițiul 19. Cookie-uri

Cerință: Parcurgeți response.cookies și afișați pentru fiecare cookie atributele .name și .secure; dacă nu există cookie-uri, afișați „Niciun cookie setat”.

[Am parcurs response.cookies și am afișat pentru fiecare cookie atributele .name și .secure. Dacă nu exista cookie-uri , programul afișează “ Nicio cookie setat”.]

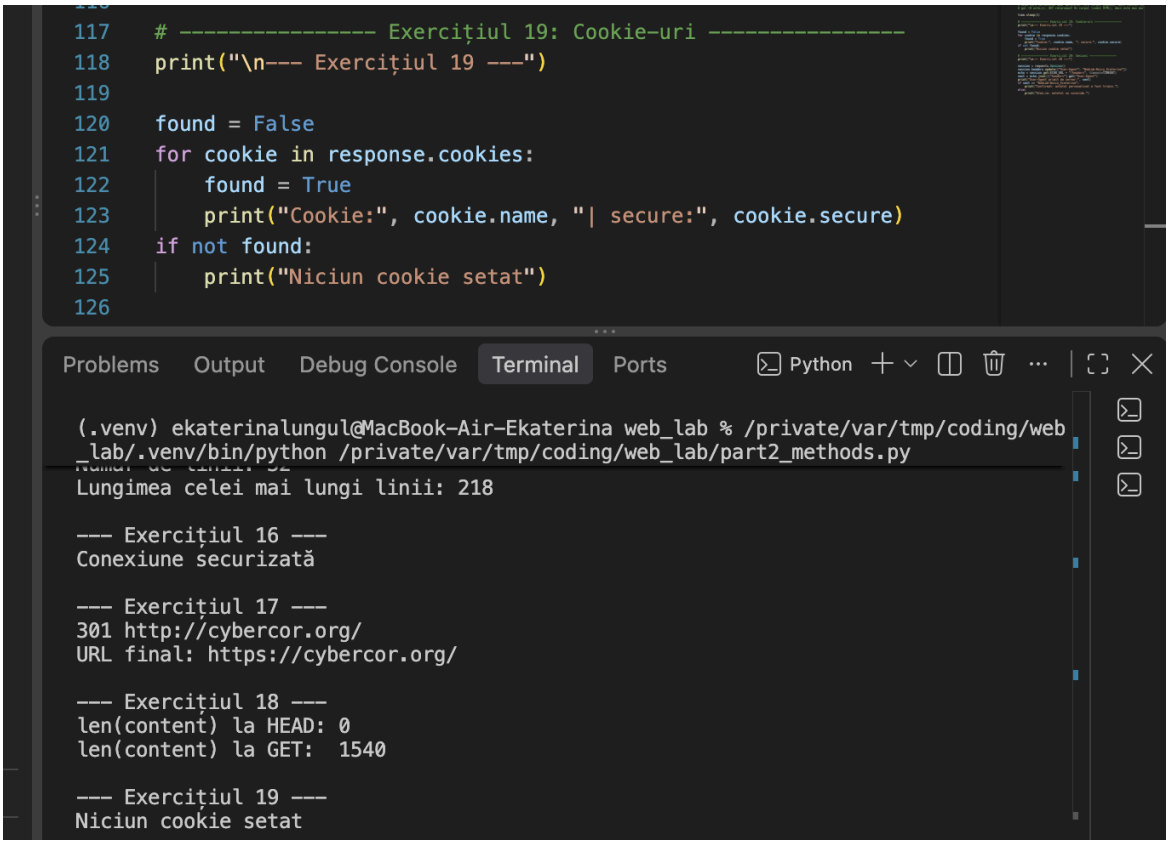


Fig.2.19.- Definirea cookie

Exercițiul 20. Sesiuni

Cerință: Creați un obiect requests.Session(), setați un User-Agent personalizat (WebLab-<numele vostru>) cu session.headers.update(), trimiteți o cerere către ECHO_URL + "/headers" și confirmați în rezultat că antetul a fost trimis.

[Am creat un obiect requests.Session() , am setat un User-Agent personalizat cu session.headers.update () si am trimis o cerere catre httpbin.org/headers. răspunsul a confirmat ca antetul a fost trimis .]

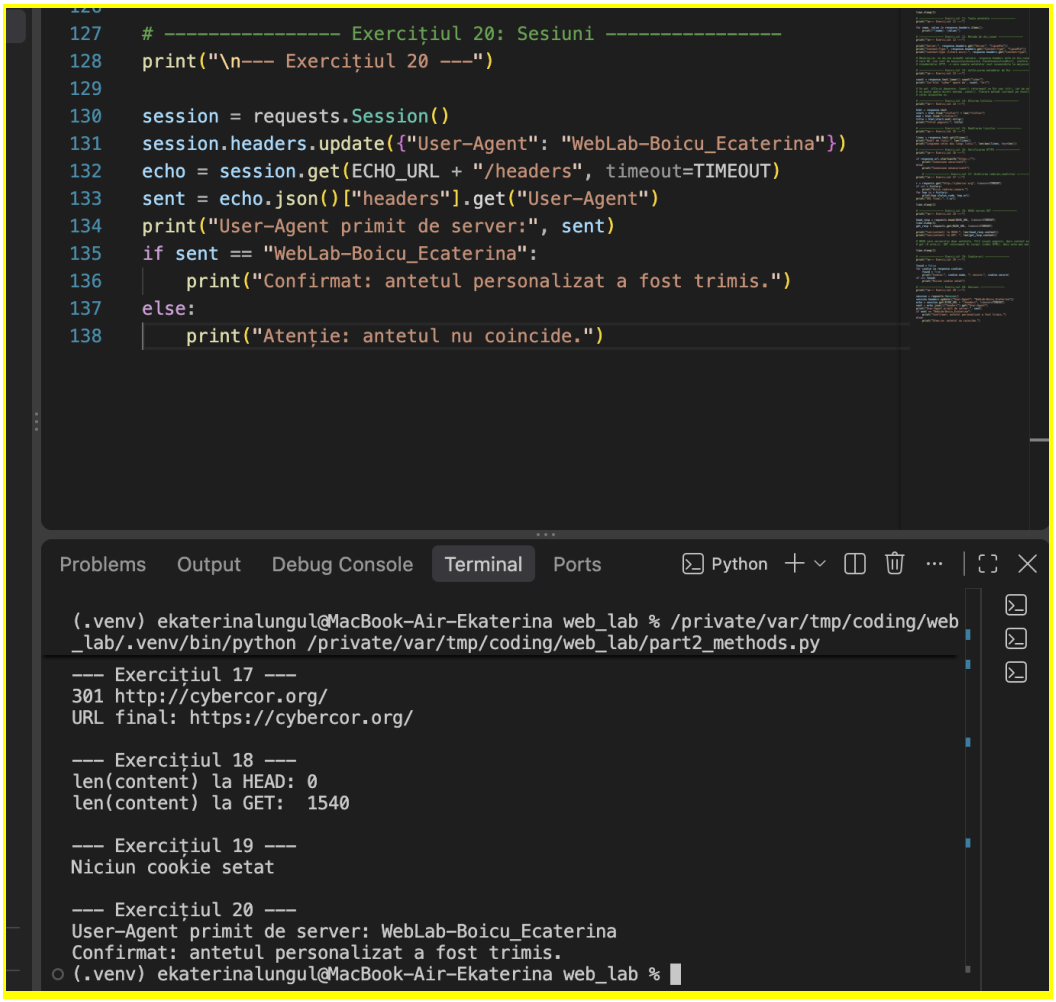


Fig 2.20. Afișarea rezultatului trimiterii antetului .

Întrebare de verificare

Întrebare: Atât response.text, cât și response.json() oferă corpul răspunsului. De ce unul are paranteze, iar celălalt nu?

[response.text este un atribut : valoarea (corpul ca sir de caractere) este deja disponibila, deci se citește fără paranteze. response.json() este o metoda: executa o acțiune (analizează corpul ca JSON și construiește un dicționar sau o lista),poate primi argumente și poate eșua ,de aceea se apelează cu paranteze.]

Partea 3. Funcții proprii

Fișier de lucru: part3_functions.py (exercițiile 21–34). Fiecare exercițiu este o funcție apelată cel puțin o dată; începând cu această parte, fiecare funcție trebuie să aibă docstring.

Exercițiul 21. Prima funcție

Cerință: Scrieți fetch(url), care returnează obiectul răspuns; apelați-o pentru BASE_URL și afișați codul de stare.

[Am scris funcția fetch(url) , care descarca pagina cu requests.get() și returnează obiectul răspuns . Am apelat-o pentru BASE_URL și am aflat codul de stare 200]

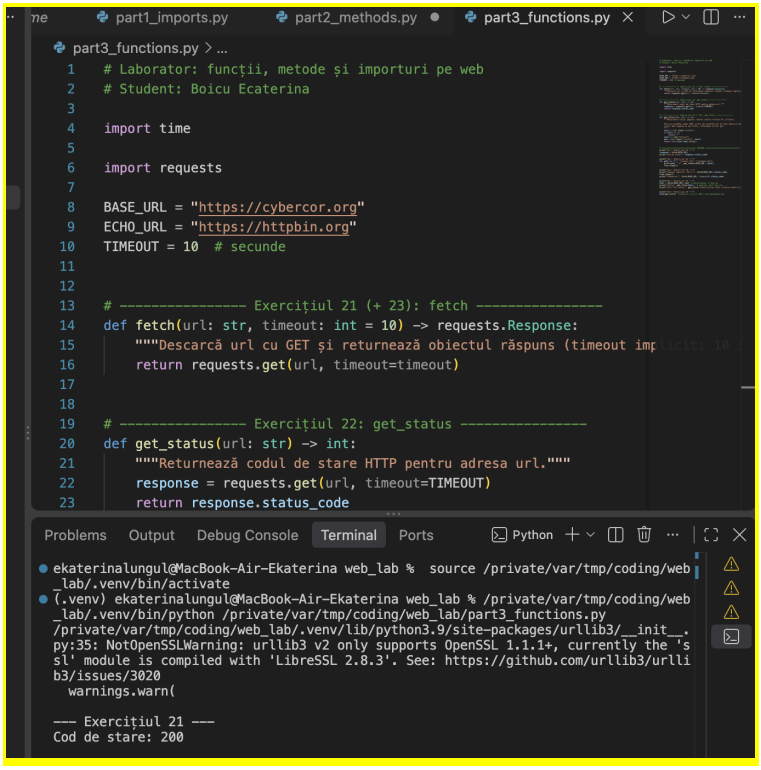


Fig.3.21 - Rezultatul itemului 21

Exercițiul 22. Returnarea unei singure valori

Cerință: Scrieți get_status(url), care returnează doar codul de stare; apelați-o pentru /, /robots.txt și /sitemap.xml, cu time.sleep(1) între apeluri.

[Am scris funcția get_status(url) , care returnează doar codul de stare. Am apelat-o pentru / ,/robots.txt si /sitemap.xml , cu time.sleep(1) între apeluri]

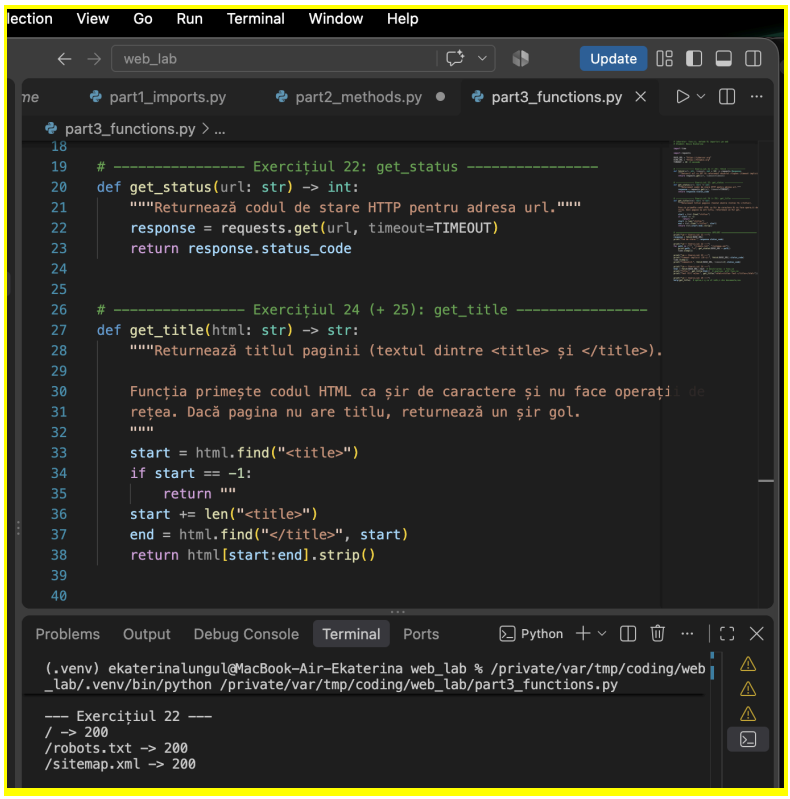


Fig.3.22- Rezultatul itemului 22

Exercițiul 23. Parametri impliciți

Cerință: Modificați funcția în fetch(url, timeout=10); apelați-o o dată cu valoarea implicită și o dată cu timeout=3.

[Am dat funcției fetch parametrul implicit timeout=10 . Am apelat-o o data cu valoarea implicită și o data cu timeout=3; în ambele cazuri cererea a reușit]

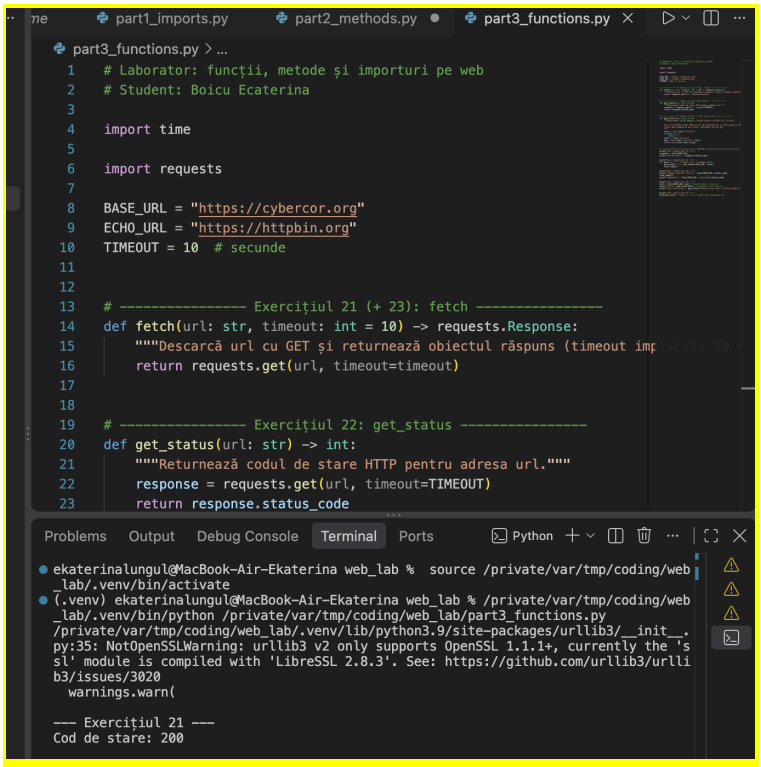


Fig.3.23.1- Itemii 21 +23

```
print("\n--- Exercițiul 23 ---")
print("timeout implicit (10 s):", fetch(BASE_URL).status_code)
time.sleep(1)
print("timeout=3:", fetch(BASE_URL, timeout=3).status_code)
```

Fig.3.23.2- Itemul 23

Exercițiul 24. O singură sarcină pe funcție

Cerință: Scrieți get_title(html), care primește un șir HTML și returnează titlul paginii (logica din exercițiul 14), fără operații de rețea.

[Am scris funcția get_title(html), care primește un șir HTML și returnează titlul paginii , folosind logica din exercițiul 14. Funcția nu face operații de rețea: descarcarcarea (fetch) și analiza (get_title]

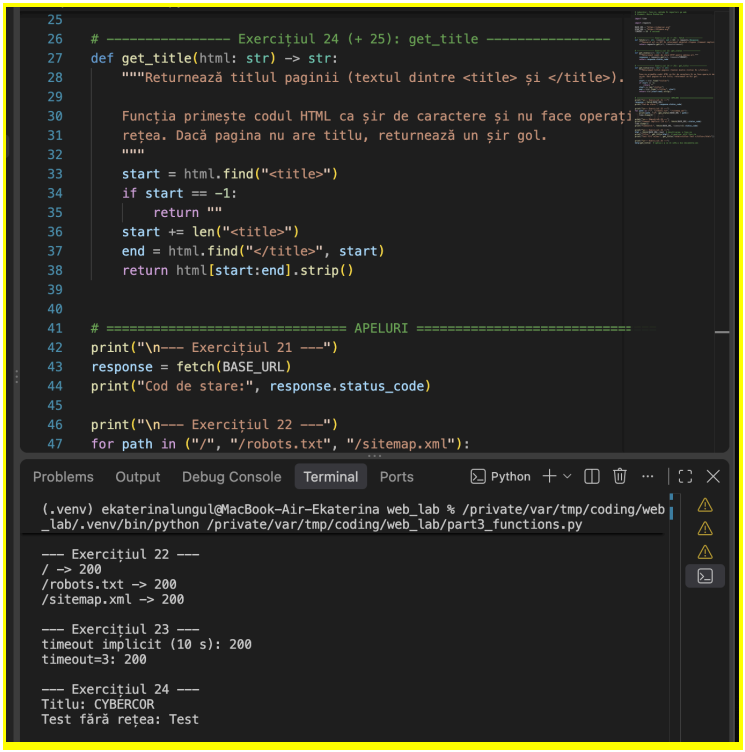


Fig.3.24- Obținerea titlului și testului

Exercițiul 25. Docstring-uri

Cerință: Adăugați un docstring funcției get_title și afișați-l cu help(get_title).

[Descrieți pașii efectuați, codul scris și rezultatele obținute.]

```
print("\n--- Exercițiul 25 ---")
help(get_title)  # apăsați q ca să ieșiți din documentație
```

Fig.3.25.1- Itemul 25 , afișarea docstring-ului cu help(get_title)

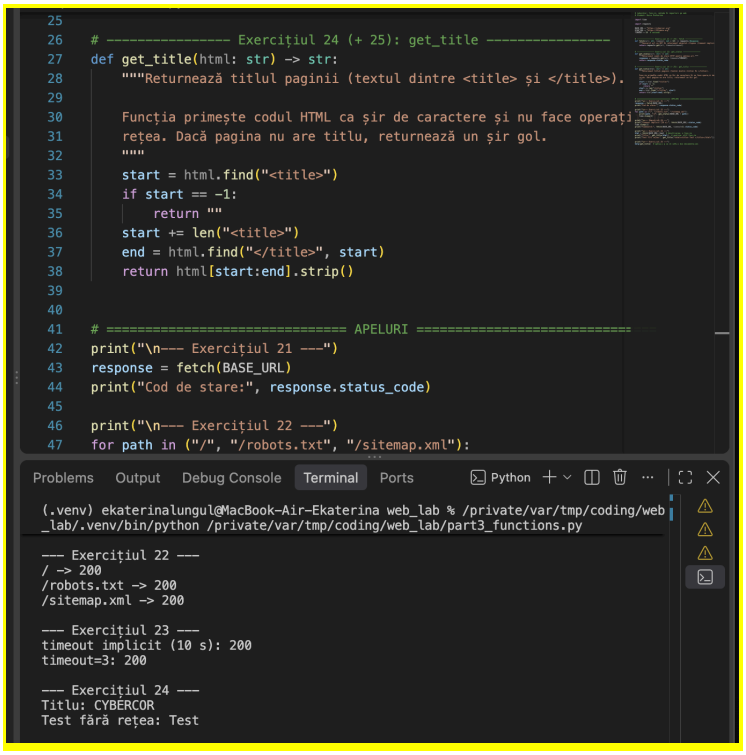


Fig.3.25.2- Itemii 24+ 25

Exercițiul 26. Adnotări de tip

Cerință: Adăugați adnotări de tip funcțiilor (de ex. def get_status(url: str) -> int:); apelați get_status(123), observați ce se întâmplă și precizați dacă adnotările vă opresc.

[Am adaugat adnotări de tip funcțiilor (de exemplu def get_status(url: str)-> int: și am apelat get_status(123), cu un număr în loc de șir.Programul a trecut de adnotare și a eșuat abia în requests, cu eroarea MissingSchema (URL invalid) , pe care am prins-o cu try / except.]

[Adnotarile de tip nu ne opresc .Ele sunt doar indicii pentru programatori și editoare , iar Python nu le verifica la rulare .]

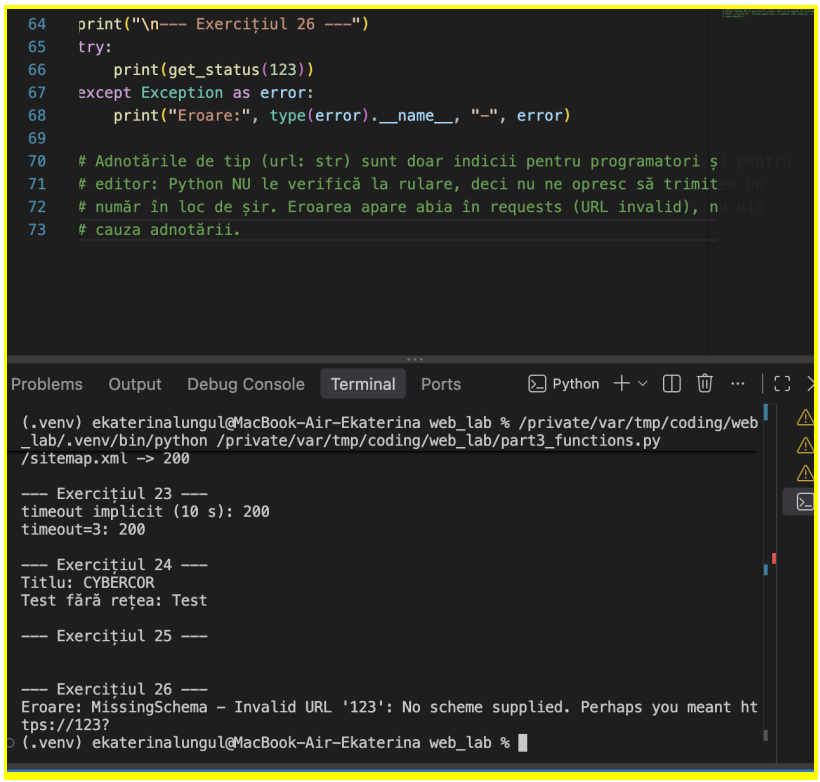


Fig.3.26- Testarea adnotarilor de tip

Exercițiul 27. Fără opriri neașteptate

Cerință: Scrieți page_exists(url), care returnează True sau False, folosind try / except requests.RequestException; verificați și pentru https://this-domain-does-not-exist.invalid.

[Am scris funcția page_exists(url),care returnează True sau False.Cererea este într-un bloc try / except request. RequestsException ,deci erorile de rețea nu opresc programul . Pentru https:/this-domain-does-not-exist.invalid a restaurant False , fără eroare]

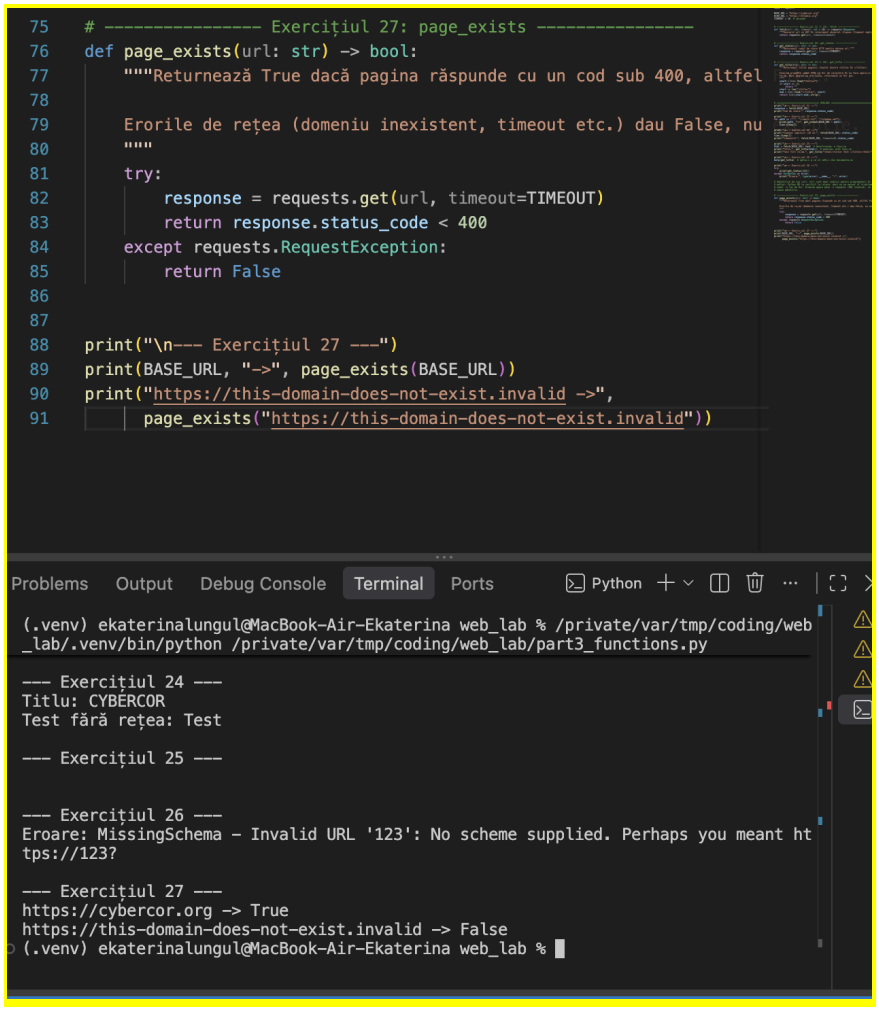


Fig.3.27- Rezultatul exercitiului 27

Exercițiul 28. Returnarea unui dicționar

Cerință: Scrieți check_paths(base, paths), care primește o listă de căi și returnează un dicționar {path: status_code}.

[Am scris funcția check_paths(base,paths) , care primește o listă de cai și returnează un dicționar { cale: cod_de_stare}. Functia asteapta o secunda intre cereri și pune None dacă o cerere eșuează. Am apelat-o pentru /, /robots.txt si /sitemap.[ml , iar rezultatul a fost dicționarul cu codurile de stare afișat în terminal.]

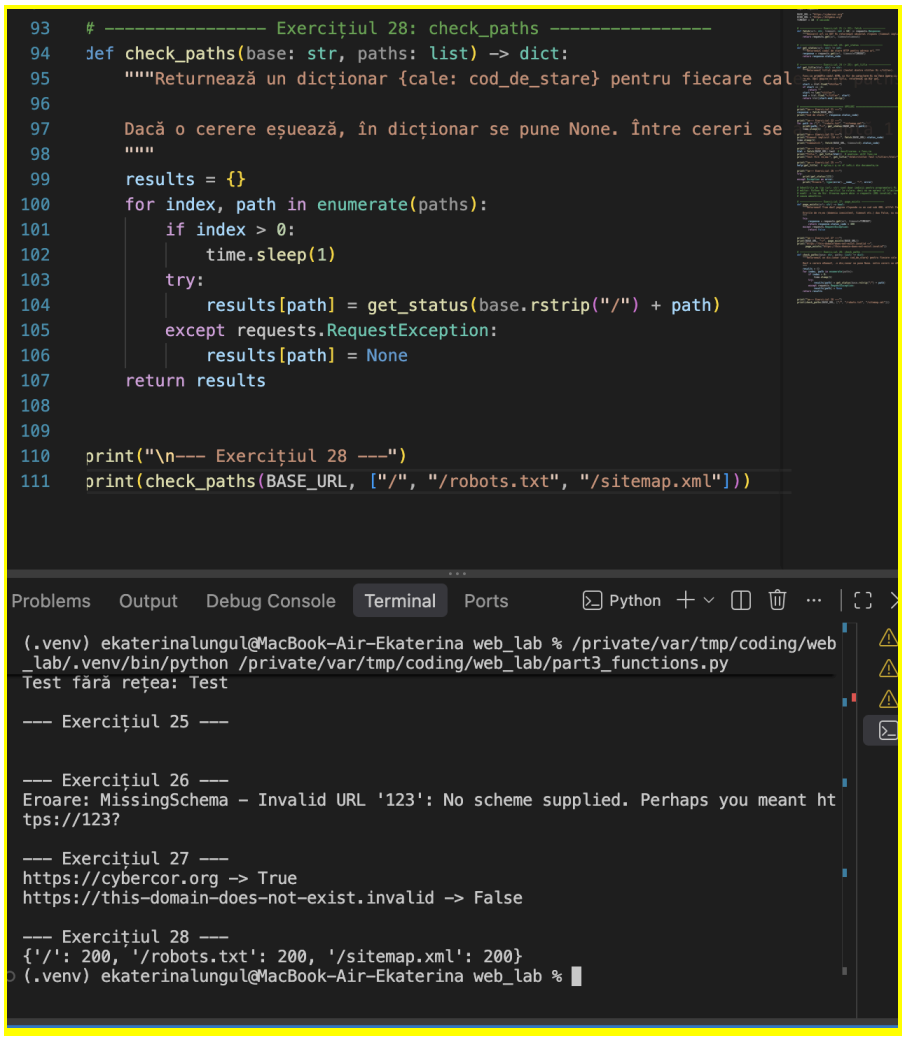


Fig.3.28- Rezolvarea itemului 28

Screenshot nr. 30

Exercițiul 29. Argumente cu nume

Cerință: Scrieți get_header(url, name, default="lipsește"), care returnează valoarea unui antet; apelați-o cu argumente cu nume: get_header(BASE_URL, name="Server").

[Am scris funcția get_header(url,name, default=”lipsește”) , care returnează valoarea unui singur antet. Am apelat-o cu argumente cu nume ; get_header (BASE_URL, name=”Server”).Pentru un antet inexistent , funcția a returnat valoarea implicită “ lipsește”.]

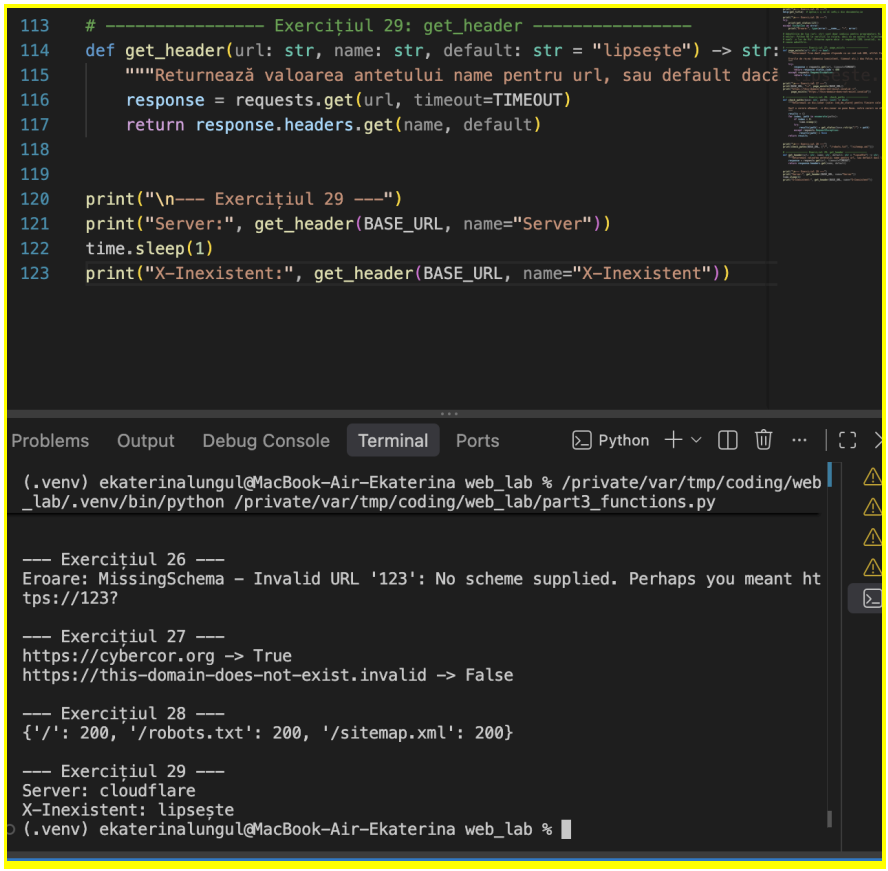


Fig.3.29 - Returnarea antetului cu denumirea argumentului

Exercițiul 30. Antete de securitate

Cerință: Scrieți security_headers(url), care verifică prezența celor cinci antete de securitate indicate în lucrare și returnează {antet: True/False}.

[Am scris funcția security_headers(url),care verifica prezenta a cinci antete de securitate (Strict-Transport-Security,content-Security-Policy,X-Frame-Options,X-Content-Type-Options ,Referrer-Policy)și returnează un dicționar {antet: True/False}.Am apelat-o pentru BASE_URL și am afisat rezultatul pentru fiecare antet.]

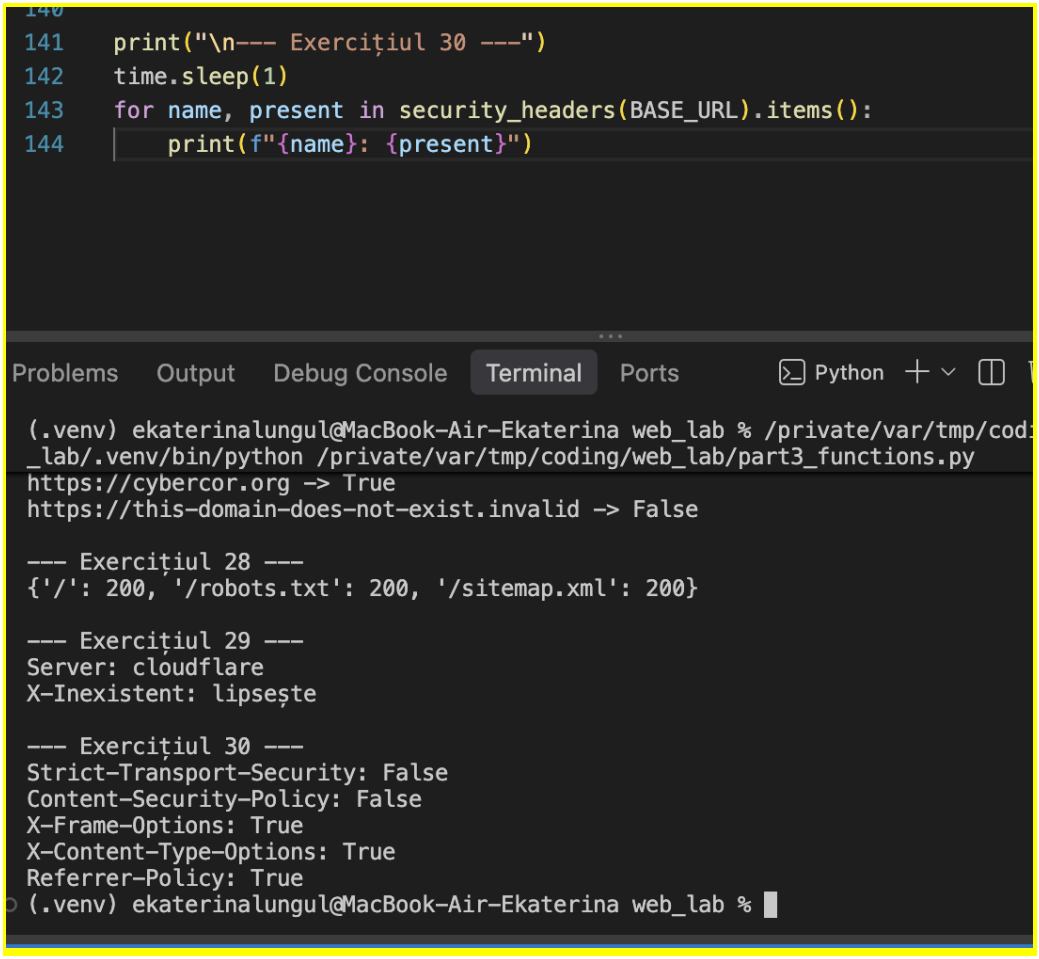


Fig.3.30- Verificarea celor cinci antete

Exercițiul 31. Funcții care apelează funcții

Cerință: Scrieți score_headers(results), care primește dicționarul din exercițiul 30 și returnează un șir de tipul „3/5”; apelați score_headers(security_headers(BASE_URL)).

[Am scris funcția score_headers(results),care primește dicționarul returnat de security_headers și returnează un șir de forma „¾”(numărul de antete prezente din total).Le-am apelat impreuna : score_headers (security_headers (BASE_URL)),iar rezultatul a fost scorul afișat în terminal.]

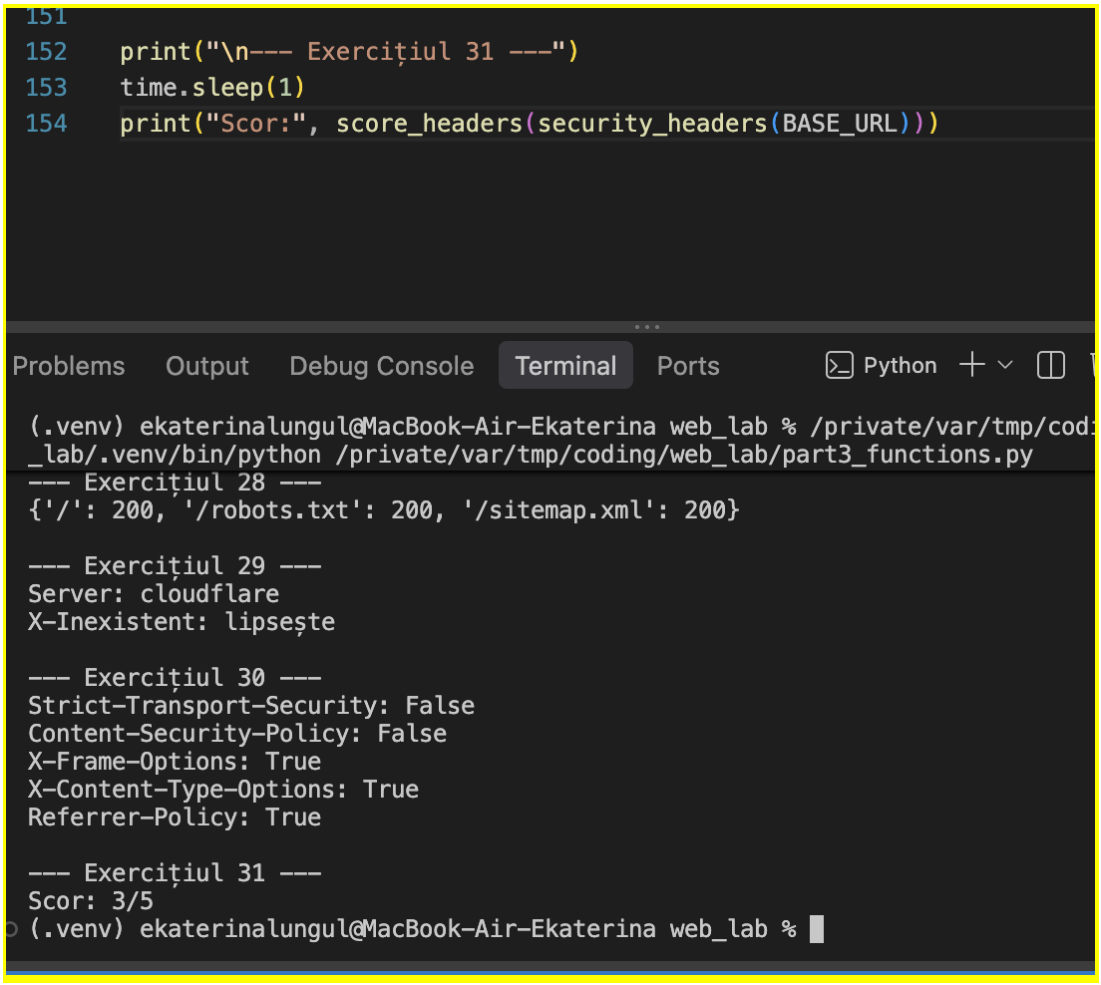


Fig.3.31- Apelarea funcției cu ajutorul funcției

Exercițiul 32. Două funcții, o sarcină

Cerință: Scrieți fetch_robots(base) (returnează textul /robots.txt sau None) și disallowed_paths(robots_text) (returnează lista valorilor Disallow:); a doua trebuie să funcționeze și pentru None.

[Am scris doua functii : fetch_robots(base),care returneaza textul fișierului /robots.txt sau None dacă lipsește , și disallowed_paths(robots_text) , care returneaza lista valorilor Disallow.A doua funcție funcționează și pentru None , caz in care returneaza o lista goala. Rezultatele sunt afișate în terminal.]

```python
---------- Exercițiul 32: fetch_robots și disallowed_paths ------
fetch_robots(base: str):
"""Returnează textul fișierului /robots.txt sau None dacă lipsește."""
try:
    response = requests.get(base.rstrip("/") + "/robots.txt", timeout
except requests.RequestException:
    return None
if response.status_code != 200:
    return None
return response.text


disallowed_paths(robots_text) -> list:
"""Returnează lista valorilor Disallow: din robots_text (listă goală
if robots_text is None:
    return []
paths = []
for line in robots_text.splitlines():
    line = line.split("#", 1)[0].strip()  # ignoră comentariile
    if line.lower().startswith("disallow:"):
        value = line.split(":", 1)[1].strip()
        if value:
            paths.append(value)
return paths
```

Fig.3.32.1- Scrierea a doua functii : returnarea textului și returnarea listei valorilor , ambele pentru None.

```
180
181
182    print("\n--- Exercițiul 32 ---")
183    time.sleep(1)
184    robots = fetch_robots(BASE_URL)
185    print("Căi interzise:", disallowed_paths(robots))
186    print("Cu None:", disallowed_paths(None))

Problems  Output  Debug Console  Terminal  Ports  Python

(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab % /private/var/tmp/cod
_lab/.venv/bin/python /private/var/tmp/coding/web_lab/part3_functions.py
Server: cloudflare
X-Inexistent: lipsește

--- Exercițiul 30 ---
Strict-Transport-Security: False
Content-Security-Policy: False
X-Frame-Options: True
X-Content-Type-Options: True
Referrer-Policy: True

--- Exercițiul 31 ---
Scor: 3/5

--- Exercițiul 32 ---
Căi interzise: []
Cu None: []
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab %
```

Fig.3.32.2- Rezultatul itemului 32

Exercițiul 33. Oricâte argumente

Cerință: Scrieți response_times(*urls), care returnează un dicționar {url: secunde}; apelați-o cu două URL-uri.

[Am scris funcția response_times(*urls), care primește oricate adrese și returnează un dicționar {url: secunde} , cu timpul măsurat prin time.perf_counter (). am apelat-o cu doua URL-uri (BASE_URL si ECHO_URL +"/get") , iar rezultatul este dicționarul afișat în terminal.]

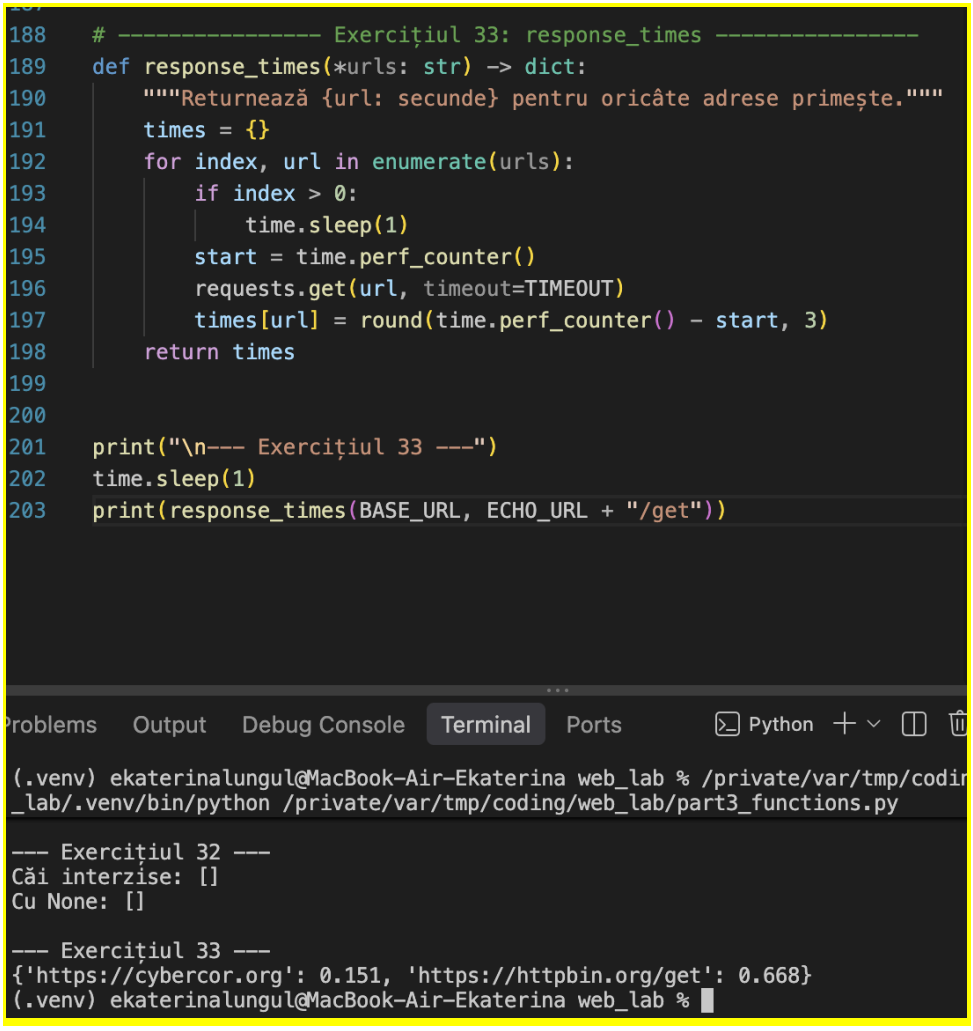


Fig.3.33- Rezultatul exercitiului 33

Exercițiul 34. Argumente cu nume colectate

Cerință: Scrieți log(message, **details), care afișează mesajul urmat de fiecare detaliu, în formatul din lucrare (de ex. verificat | url=... | status=200).

[Am scris funcția log(message,**details),care afișează mesajul urmat de fiecare detaliu primit ca argument cu nume (**details le colectează într-un dictionar) . Apelul

log(“verificat,url=BASE_URL,status=200)a afișat verificat | url=https://cybercor.org | status=200.]

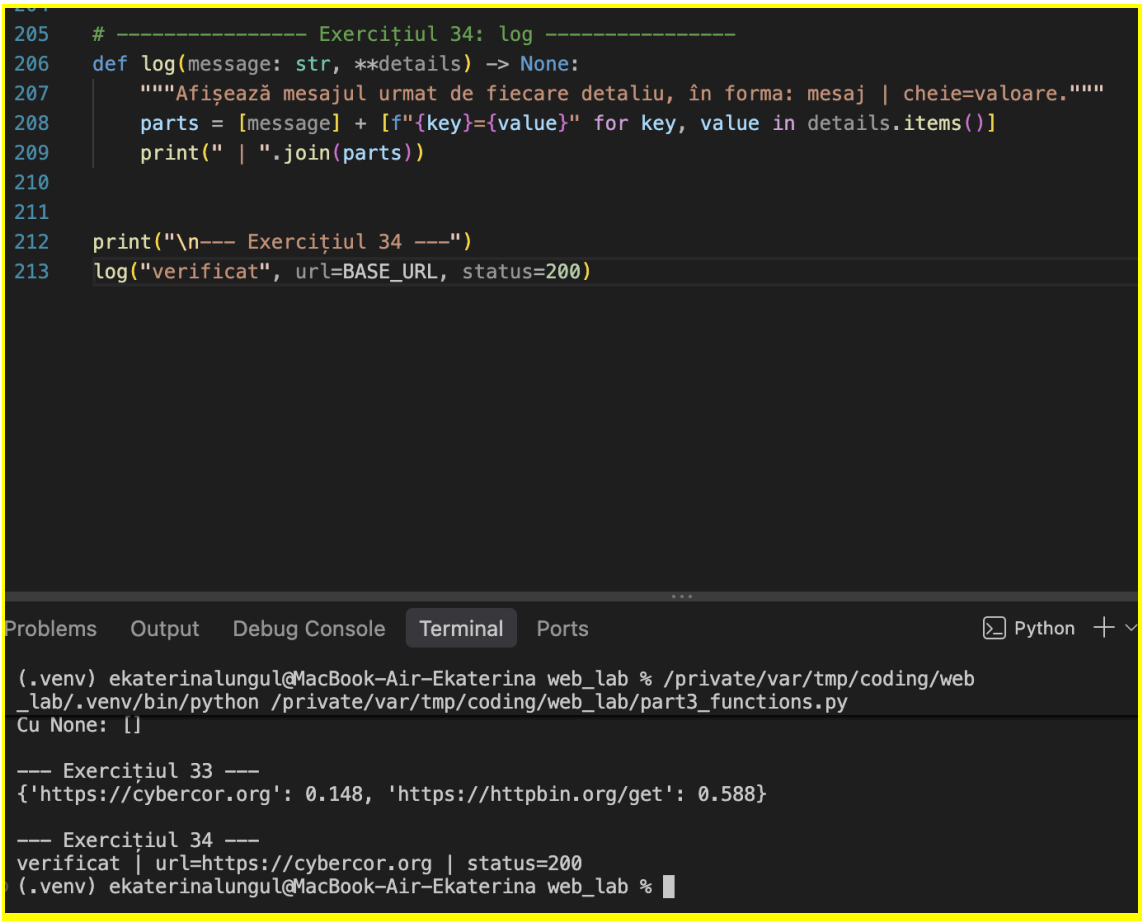


Fig.3.34 - Afișarea mesajului

Întrebare de verificare

Întrebare: Care este diferența dintre a afișa o valoare cu print() în interiorul unei funcții și a o returna cu return?

[print() doar afișează valoarea de pe ecran , iar funcția returnează None , deci apelantul nu o poate folosi mai departe . return trimite valoarea înapoi apelantului , care o poate stoca , testa sau transmite altei funcții (de exemplu score_headers(security_headers(...))). Funcțiile care returnează sunt reutilizabile.]

Partea 4. Module standard

Fișier de lucru: part4_modules.py (exercițiile 35–43). Fiecare soluție se scrie într-o funcție.

Exercițiul 35. Descompunerea unui URL

Cerință: Aplicați urllib.parse.urlparse() pe https://cybercor.org/path?x=1#top și afișați .scheme, .netloc, .path, .query și .fragment.

[Am scris funcția describe_url(url) , care descompune un URL cu urllib.parse.urlparse() și afișează componentele .scheme , .netloc,.path,.query și .fragment . Am aplicat-o pe https://cybercor.org/path?x=1#top, iar rezultatul a fost : https,cybercor.org,/path,x=1,top.]

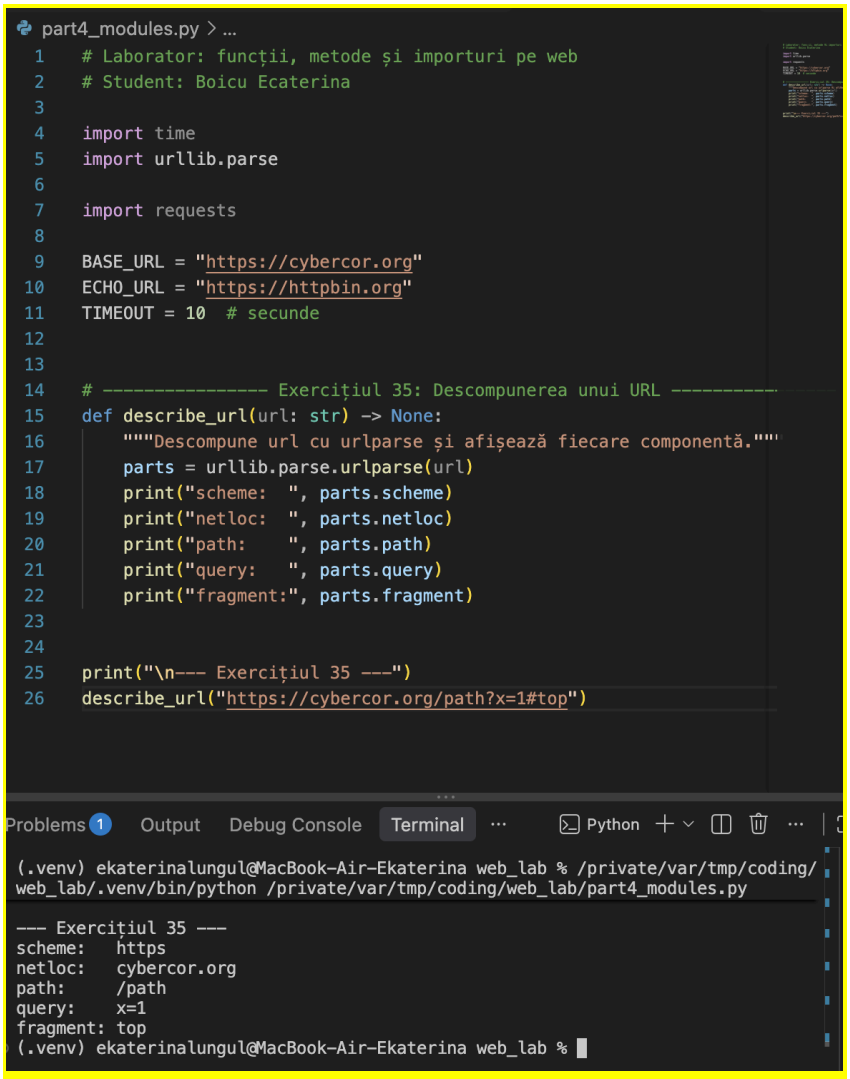


Fig.4.35 - Descompunerea URL

Exercițiul 36. Compunerea URL-urilor

Cerință: Folosiți urllib.parse.urljoin() pentru a transforma legăturile relative /about, contact.html și ../index.html în URL-uri complete, pornind de la BASE_URL.

[Am scris funcția make.absolut(base,links), care folosește urllib.parse.urljoin() pentru a transforma legăturile relative /about , contact.html și ../index.html in URL-uri complete,porninddelaBASE_URL.Rezultatul:https://cybercor.org/about,https://cybercor.org/contact.htmlsi https://cybercor.org/index.html.]

```
28  # ---------------- Exercițiul 36: Compunerea URL-urilor ------------
29  def make_absolute(base: str, links: list) -> list:
30      """Transformă legăturile relative în URL-uri complete, pornind de la
31      return [urllib.parse.urljoin(base, link) for link in links]
32
33
34  print("\n--- Exercițiul 36 ---")
35  relative = ["/about", "contact.html", "../index.html"]
36  for link, full in zip(relative, make_absolute(BASE_URL, relative)):
37      print(f"{link:15} -> {full}")
```

Problems 1 Output Debug Console Terminal ··· Python

```
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab % /private/var/tmp/coding/
web_lab/.venv/bin/python /private/var/tmp/coding/web_lab/part4_modules.py
query:    x=1
fragment: top

--- Exercițiul 36 ---
/about          -> https://cybercor.org/about
contact.html    -> https://cybercor.org/contact.html
../index.html   -> https://cybercor.org/index.html
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab %
```

Fig.4.36 – Rezultatul transformării legăturilor relative în URL

Exercițiul 37. Extragerea legăturilor

Cerință: Cu re.findall() obțineți legăturile (href) de pe pagina principală și puneți codul în extract_links(html), care returnează legăturile fără duplicate.

[Am scris funcția extract_links(html) , care folosește re.findall (r'href="[^"]+)" ' , html) pentru a extrage legăturile (href) din pagina și le returneaza fara duplicate , in ordinea aparitiei.Am apelat-o pe pagina principala, iar rezultatul este lista legăturilor unică afișată în terminal.]

```python

print("\n--- Exercițiul 37 ---")
page = requests.get(BASE_URL, timeout=TIMEOUT).text
links = extract_links(page)
print(len(links), "legături unice:")
for link in links:
    print(" ", link)

time.sleep(1)
```

```
Problems 1  Output  Debug Console  Terminal  ···  Python

(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab % /private/var/tmp/coding/
web_lab/.venv/bin/python /private/var/tmp/coding/web_lab/part4_modules.py
/about          -> https://cybercor.org/about
contact.html    -> https://cybercor.org/contact.html
../index.html   -> https://cybercor.org/index.html

--- Exercițiul 37 ---
1 legături unice:
  /assets/index-DlIqjleG.css
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab %
```

Fig.4.37 - Afișarea legăturilor unice de pe pagina principala

Exercițiul 38. Interne sau externe?

Cerință: Scrieți split_links(links, domain), care returnează două liste: legăturile de pe același domeniu și legăturile către alte site-uri (urljoin(), apoi urlparse().netloc).

[Am scris funcția split_links(links, domain) , care returneaza doua liste: legăturile de pe același domeniu și legăturile către alte site-uri.Mai intai transforma legăturile relative în adrese complete cu urljoin(), apoi compara urlparse().netloc cu domeniul.Am aplicat-o pe legaturile paginii principale, iar rezultatul (listele cu legături interne și externe) este afișat in terminal.]

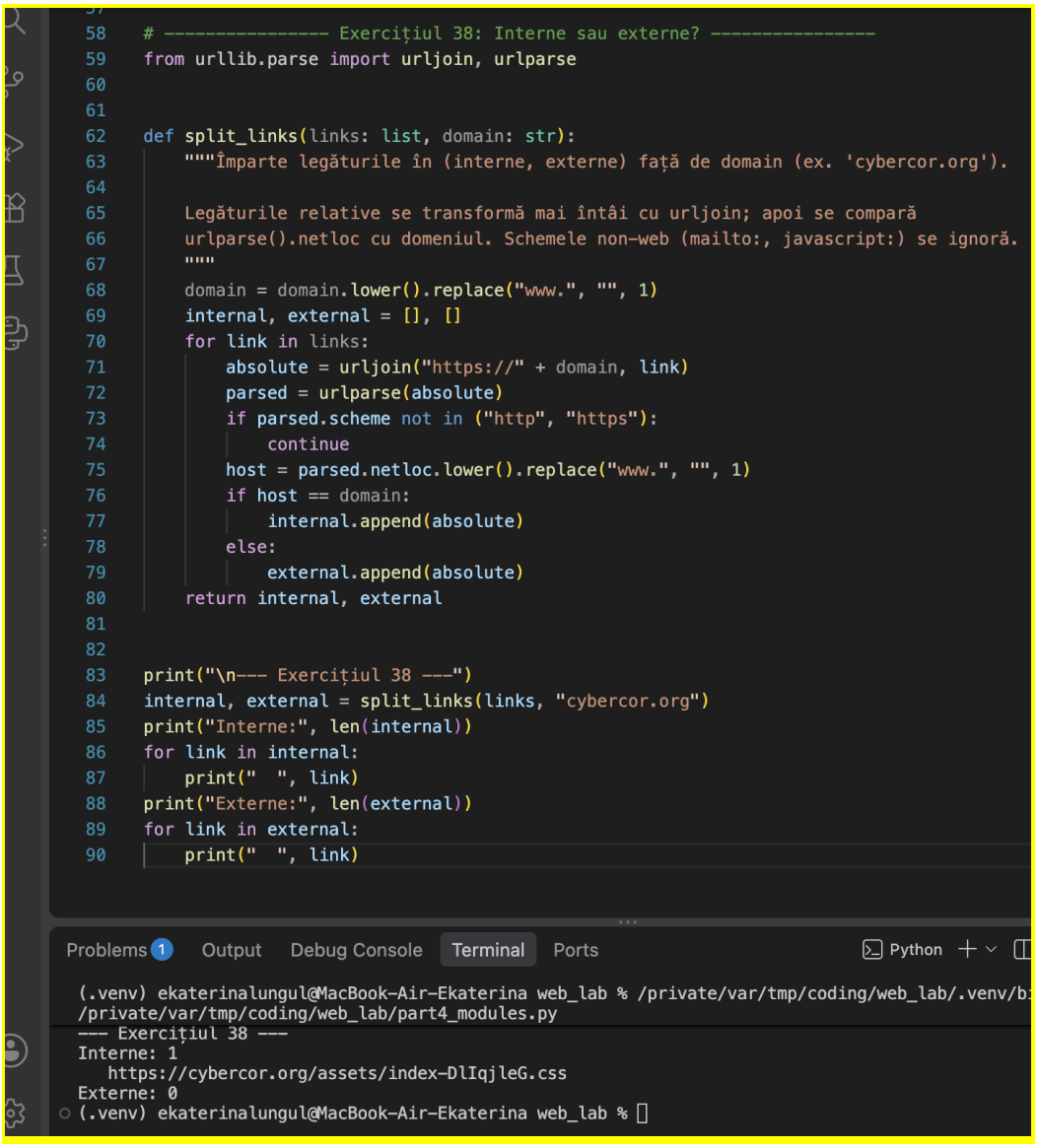


Fig.4.38- Rezultatul itemului 38

Exercițiul 39. Propriile metode

Cerință: Creați o clasă care moștenește html.parser.HTMLParser și suprascrie handle_starttag(), pentru a colecta atributul src al fiecărui tag <img>. Testați mai întâi pe fragmentul HTML din lucrare (assert), apoi pe site-ul real; verificați rezultatul prin comparație cu response.text.lower().count("<img") și cu codul sursă al paginii (Ctrl+U).

[Am creat clasa ImageFinder , care mosteneste html.parser.HTMLParser si suprascrie metoda handle_startag() pentru a colecta atributul src al fiecărui tag<img>.Am testat-o mai intai pe un fragment HTML cunoscut cu assert , iar testul a trecut. Apoi am rulat-k pe pagina reala si am comparat numărul de imagini găsite page.lower().count(“<img”); Pagina nu are taguri

<img> , apare 0 imagini găsite și =o. E în regula , lucrarea spune explicit ca o lista goala nu e neaparat o greșeală , căci unele site-uri afișează imaginile prin CSS sau SVG (CSS definește *cum arată* elementele, în timp ce SVG este un tip de *imagine* pe care o poți afișa pe ecran.)]

```
4_modules.py > ...
    # ---------------- Exercițiul 39: Propriile metode (HTMLParser) ----------------
from html.parser import HTMLParser


class ImageFinder(HTMLParser):
    """Parser HTML care colectează atributul src al fiecărui tag <img>."""

    def __init__(self):
        """Pregătește lista goală în care se vor colecta adresele imaginilor."""
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        """Apelată automat de parser la fiecare tag de deschidere."""
        if tag == "img":  # HTMLParser dă numele tag-urilor cu litere mici
            src = dict(attrs).get("src")
            if src:
                self.images.append(src)


print("\n--- Exercițiul 39 ---")

# Testul pe HTML cunoscut
test_html = """
<html><body>
<img src="/logo.png" alt="Logo">
<IMG SRC="poza.jpg">
<img alt="imagine fără src">
<img src="https://cdn.example.com/banner.webp" />
<a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)
assert finder.images == [
```

Fig.4.39.1 - Testarea intai pe fragmentul HTML

```
part4_modules.py > ...
118     <IMG SRC="poza.jpg">
119     <img alt="imagine fără src">
120     <img src="https://cdn.example.com/banner.webp" />
121     <a href="/despre">Aceasta nu este o imagine</a>
122     </body></html>
123     """
124
125     finder = ImageFinder()
126     finder.feed(test_html)
127     print(finder.images)
128     assert finder.images == [
129         "/logo.png",  # imagine obișnuită
130         "poza.jpg",  # tag scris cu majuscule
131         "https://cdn.example.com/banner.webp",  # tag care se închide singur
132     ], "Parserul nu a găsit exact imaginile așteptate"
133     print("Testul a trecut!")
134
135     # Rulare pe pagina reală (folosim codul HTML descărcat la exercițiul 37)
136     finder = ImageFinder()
137     finder.feed(page)
138     print(len(finder.images), "imagini găsite")
139     for src in finder.images:
140         print(src)
141     print('Verificare: page.lower().count("<img") =', page.lower().count("<img"))

Problems 1  Output  Debug Console  Terminal  Ports

(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab % /private/var/tmp/coding/web_lab/.
/private/var/tmp/coding/web_lab/part4_modules.py
  /assets/index-DlIqjleG.css

--- Exercițiul 38 ---
Interne: 1
   https://cybercor.org/assets/index-DlIqjleG.css
Externe: 0

--- Exercițiul 39 ---
['/logo.png', 'poza.jpg', 'https://cdn.example.com/banner.webp']
Testul a trecut!
0 imagini găsite
Verificare: page.lower().count("<img") = 0
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab %
```

Fig.4.39.2 - Verificarea rezultatului prin comparare; Rezultatul final

Exercițiul 40. Amprenta paginii

Cerință: Scrieți page_fingerprint(url), care returnează hash-ul SHA-256 al conținutului; rulați-o de două ori, comparați rezultatele și explicați când ar fi diferite.

[Am scris funcția page_fingerprint(url),care returnează hash-ul SHA-256 al conținutului paginii,calculat cu hashlib.sha256(response.content).hexdigest().Am rulat-o de doua ori pe pagina principala si am comparat rezultatele; ele sunt afișate în terminal .]

[Amprentele ar fi diferite dacă pagina s-ar schimba între cele doua cereri:conținutul dinamic (data/ora,contor de vizite, token CSRF(este o valoare secretă, unică și aleatorie generată de server și atașată unui formular sau cereri web, care dovedește că acțiunea a fost inițiată chiar de utilizator și nu printr-un atac trimis de un site malițios),reclame) sau actualizarea paginii de către administrator.Chiar și un singur octet diferit schimba complet hash-ul..]

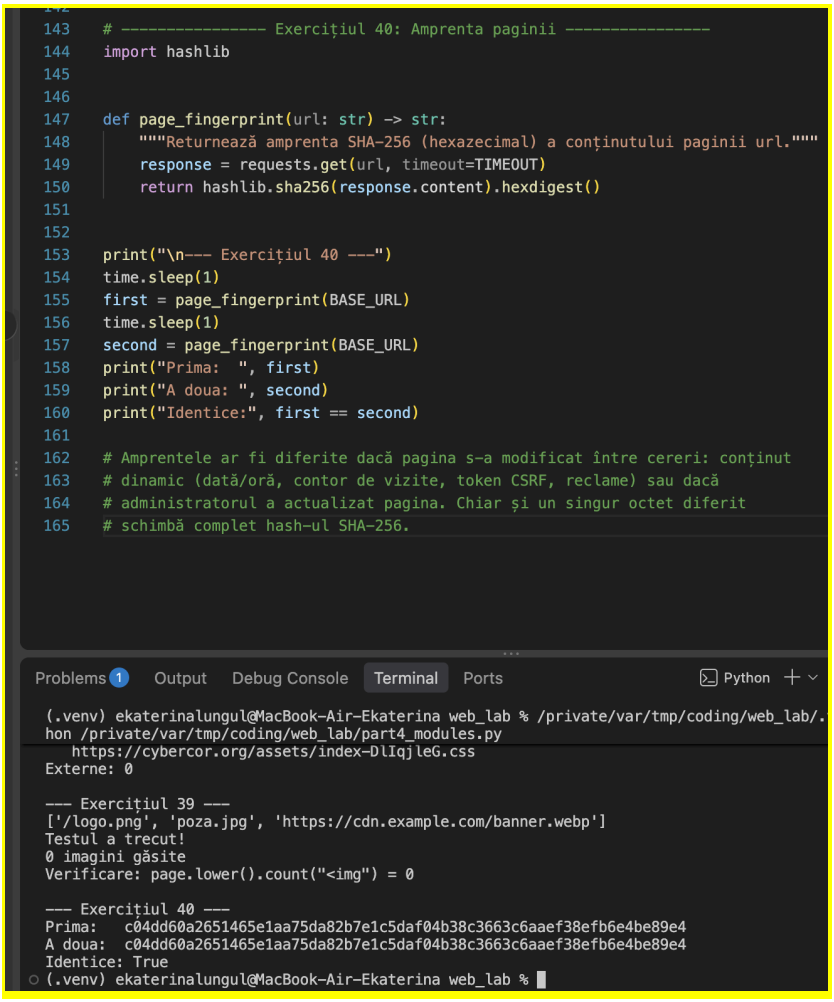


Fig.4.40 - Afișarea amprentei paginii (pagina este statică- indicele True)

Exercițiul 41. Salvarea în JSON

Cerință: Salvați antetele paginii principale în headers.json (json.dump, indent=2); apoi încărcați fișierul înapoi cu json.load() și afișați un antet din el.

[Am scris functia save_headers(url) , care salveaza antetele paginii principale in headers.json cu json.dump(dict(response.headers),file, indent=2).Apoi am încărcat fișierul înapoi cu json.load() și am aflat un antet din el (Content-Type).Fișierul headers.json a fost creat în directorul web_lab.]

```
167  # ---------------- Exercițiul 41: Salvarea în JSON ----------------
168  import json
169
170
171  def save_headers(url: str, filename: str = "headers.json") -> dict:
172      """Salvează antetele paginii url în filename (JSON) și returnează ce s-a salvat."
173      response = requests.get(url, timeout=TIMEOUT)
174      with open(filename, "w", encoding="utf-8") as file:
175          json.dump(dict(response.headers), file, indent=2)
176      with open(filename, encoding="utf-8") as file:
177          return json.load(file)
178
179
180  print("\n--- Exercițiul 41 ---")
181  time.sleep(1)
182  data = save_headers(BASE_URL)
183  print("Salvate", len(data), "antete în headers.json")
184  ctype = next((v for k, v in data.items() if k.lower() == "content-type"), "lipsește")
185  print("Content-Type din fișier:", ctype)
```

```
Problems 1   Output   Debug Console   Terminal   Ports                Python

(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab % /private/var/tmp/coding/web_lab/.venv/
hon /private/var/tmp/coding/web_lab/part4_modules.py
['/logo.png', 'poza.jpg', 'https://cdn.example.com/banner.webp']
Testul a trecut!
0 imagini găsite
Verificare: page.lower().count("<img") = 0

--- Exercițiul 40 ---
Prima:   c04dd60a2651465e1aa75da82b7e1c5daf04b38c3663c6aaef38efb6e4be89e4
A doua:  c04dd60a2651465e1aa75da82b7e1c5daf04b38c3663c6aaef38efb6e4be89e4
Identice: True

--- Exercițiul 41 ---
Salvate 18 antete în headers.json
Content-Type din fișier: text/html; charset=utf-8
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab %
```

Fig.4.41- Afișarea de N antete (în cazul meu N=18)

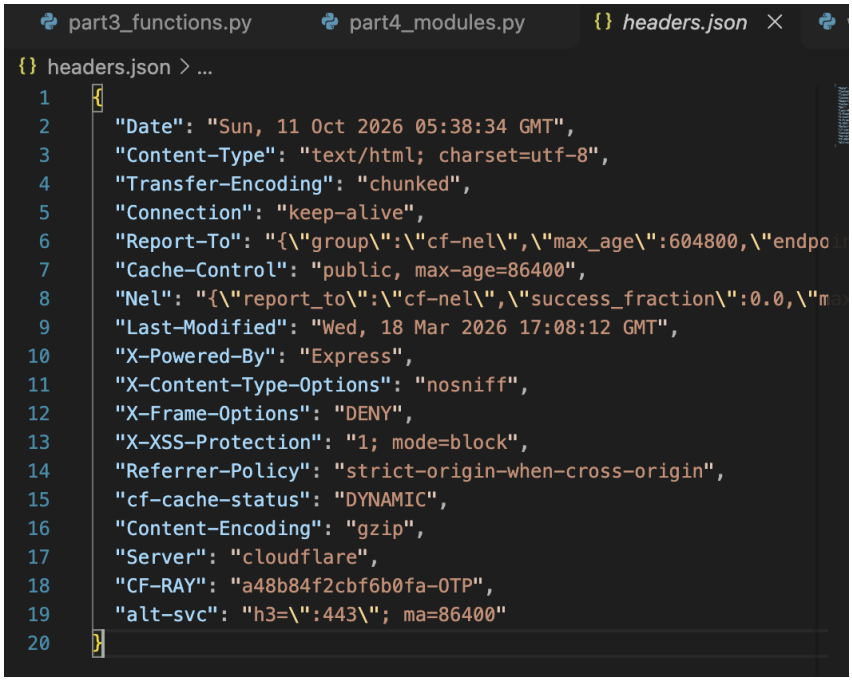


Fig.4.41.1- Fișierul headers.json

Exercițiul 42. Interogare DNS

Cerință: Scrieți resolve(hostname), care returnează adresa IP (socket.gethostbyname()); afișați adresa IP a domeniului cybercor.org.

[Am scris funcția resolve(hostname),care returnează adresa IP a unui domeniu cu socket.gethostbyname().Am apelat-o pentru domeniul cybercor.org,iar adresa IP obtinuta este afișată în terminal]

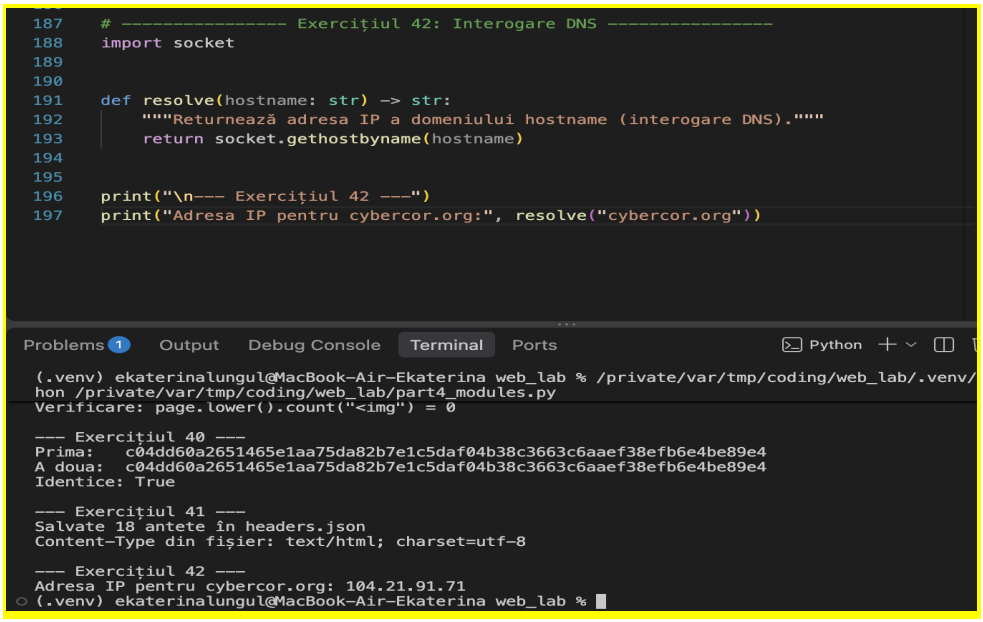


Fig.4.42- Rezultatul exercitiului 42

Exercițiul 43. Expirarea certificatului

Cerință: Scrieți cert_days_left(hostname), care se conectează la portul 443, citește certificatul și returnează numărul de zile rămase până la expirare.

[Am scris funcția cert_days_left(hostname) ,care se conecteaza la portul 443 ,citeste certificatul cu getpeercert() si returneaza numarul de zile rămase până la expirare ,calculat din campul notAfter (convertit cu ssl.cert_time_to_seconds().Am apelat-o pentru cybercor.org ,iar rezultatul este afisat in terminal.]

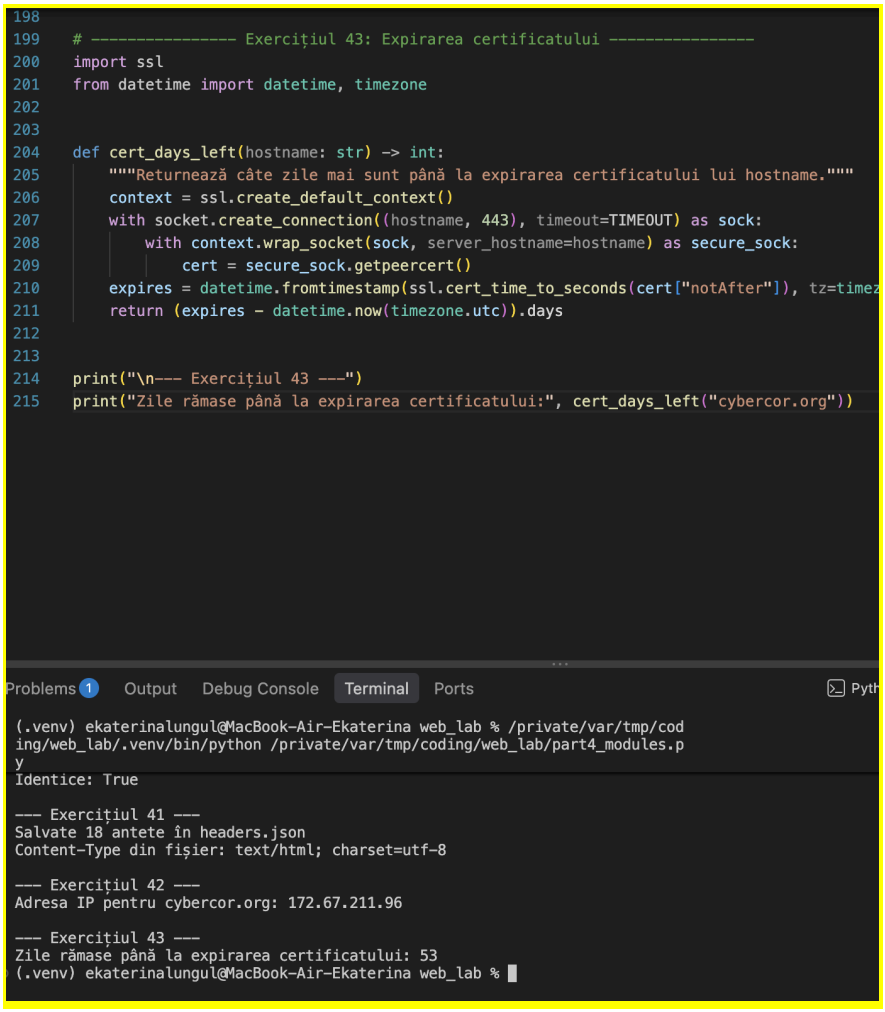


Fig.4.43- Afișarea rezultatului expirării certificatului

Întrebare de verificare

Întrebare: În exercițiul 39, cine apelează handle_starttag()? (voi nu o apelați niciodată direct)

[handle_starttag() este apelată automat de parser ,nu de noi.Când apelăm feed(html) ,HTMLParser parcurge textul și , la fiecare tag de deschidere întâlnit ,apelează singur metoda handle_starttag(tag,attrs).Noi doar o suprascriem ,ca sa stabilim ce se întâmplă la fiecare tag.]

Partea 5. Modulul propriu

Fișier de lucru: webtools.py, main.py (exercițiile 44–50).

Exercițiul 44. Crearea unui modul

Cerință: Mutați funcțiile fetch, get_status, get_title și security_headers în webtools.py; în main.py importați webtools și apelați webtools.get_title(webtools.fetch(BASE_URL).text).

[Am mutat funcțiile fetch,get_status,get_title și security_headers intr-un fisier nou , webtools.py ,care devine modulul meu propriu,In main.py am scris import webtools si am apelat webtools.get_title(webtools.fetch(BASE_URL).text).rezultatul este titlul paginii (cel din exercițiul 24) ,afișat în terminal.]

webtools.py > security_headers

```python
# Laborator: funcții, metode și importuri pe web
# Student: Boicu Ecaterina
"""webtools — modulul propriu cu funcțiile reutilizabile pentru analiza unui site."""

import requests

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

SECURITY_HEADERS = [
    "Strict-Transport-Security",
    "Content-Security-Policy",
    "X-Frame-Options",
    "X-Content-Type-Options",
    "Referrer-Policy",
]


def fetch(url: str, timeout: int = TIMEOUT) -> requests.Response:
    """Descarcă url cu GET și returnează obiectul răspuns."""
    return requests.get(url, timeout=timeout)


def get_status(url: str) -> int:
    """Returnează codul de stare HTTP pentru adresa url."""
    return fetch(url).status_code


def get_title(html: str) -> str:
    """Returnează titlul paginii (textul dintre <title> și </title>), sau '' dacă lipsește."""
    start = html.find("<title>")
    if start == -1:
        return ""
    start += len("<title>")
    end = html.find("</title>", start)
    return html[start:end].strip()


def security_headers(url: str) -> dict:
    """Returnează {antet: True/False} pentru cele 5 antete de securitate."""
    response = fetch(url)
    return {name: name in response.headers for name in SECURITY_HEADERS}
```

Fig.5.44.1 - Mutarea functiilor in webtools.py

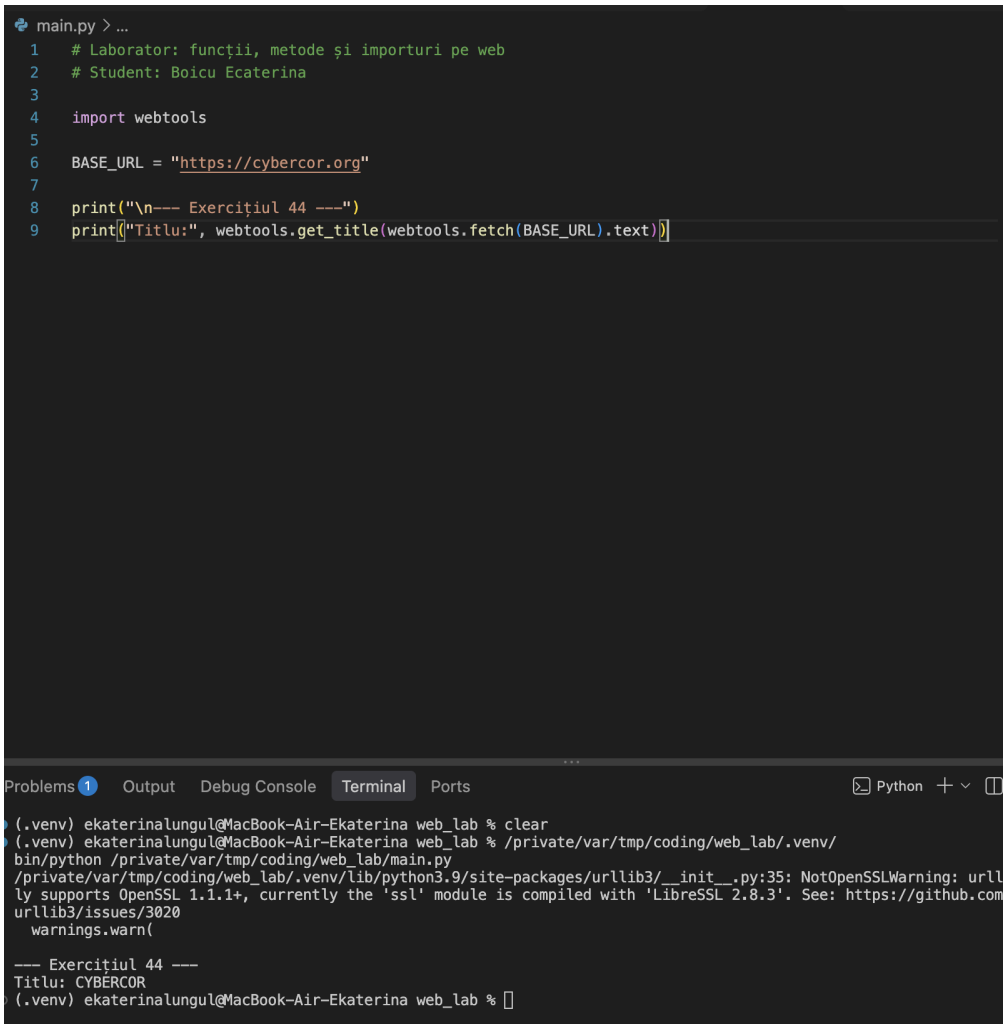


Fig.5.44.2 - Implementarea itemului 44 in fisierul main.py -importul webtools

Exercițiul 45. Importarea anumitor nume

Cerință: În main.py treceți la from webtools import get_title, security_headers; explicați într-un comentariu ce s-ar întâmpla dacă main.py ar avea și o funcție proprie get_title.

[In main.py am trecut la from webtools import get_title,securty_headers ,deci funcțiile se apelează direct ,fara prefixul webtools.Am apelat get_title, security_headers,iar rezultatele sunt afișate în terminal.]

[Daca main.py ar avea si o functie proprie get_title,ar exista doua nume identice în același spațiu de nume.Ultima definiție sau atribuire o suprascrie pe cealaltă , fără eroare , deci una dintre funcții ar fi pierdută .Conflictul se evita cu import webtools (apel webtools..get_title) sau cu un alias.]

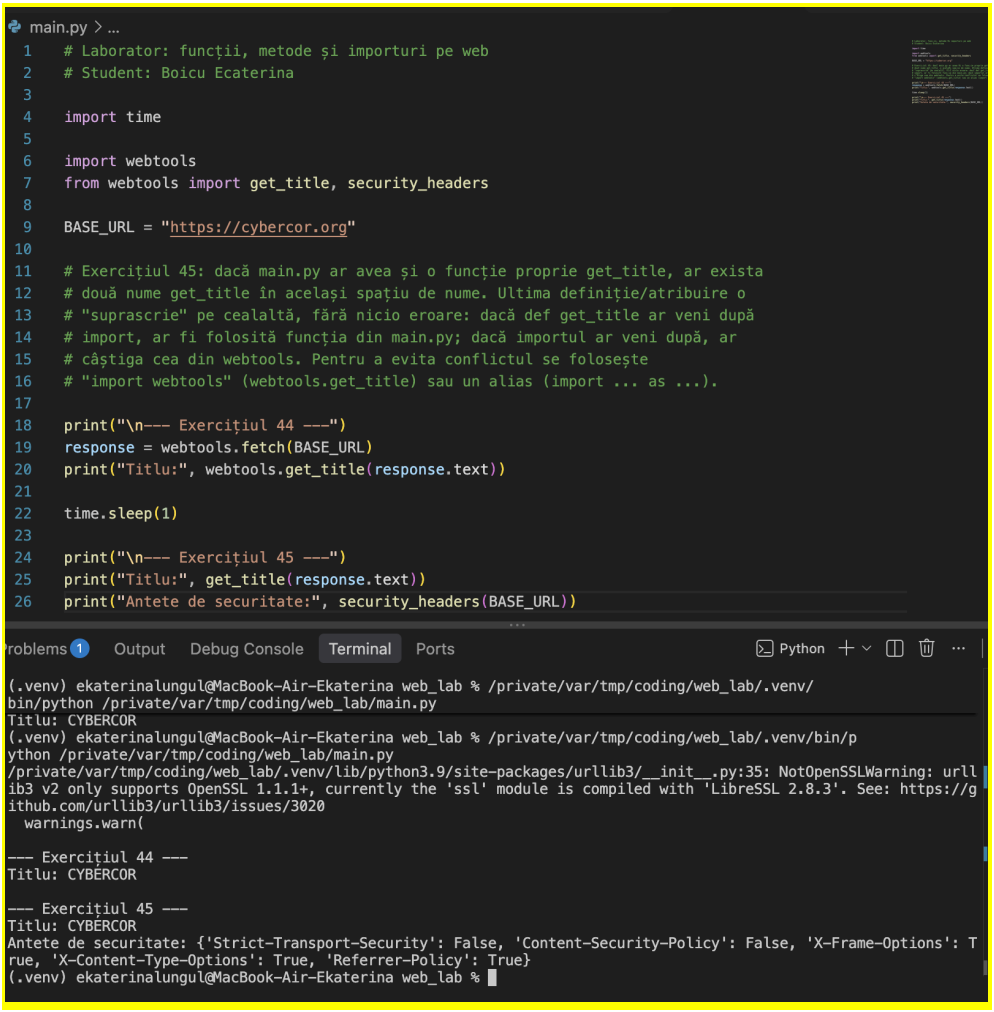


Fig.5.45- Apelarea funcțiilor fără prefixul webtools

Exercițiul 46. Blocul main

Cerință: Adăugați la finalul webtools.py blocul if __name__ == "__main__" cu autotestul din lucrare; rulați python webtools.py, apoi python main.py și explicați de ce autotestul rulează doar în primul caz.

[Am adaugat la finalul fisierului webtools.py blocul if __name__==”__main__”:,care afișează un autotest cu get_status(“https://cybercor.org”).Am rulat python webtools.py ,unde autotestul a afisat Autotest:200, apoi python main.py ,unde autotestul nu a apărut.]

[Cand rulam direct python webtools.py ,variabila __name__ este “__main__” , deci blocul se executa.Cand main.py face import webtools, __name__ devine “webtools” , deci condiția e falsă și autotestul este sărit.Astfel codul de test rulează doar la execuția directă,nu la import.]

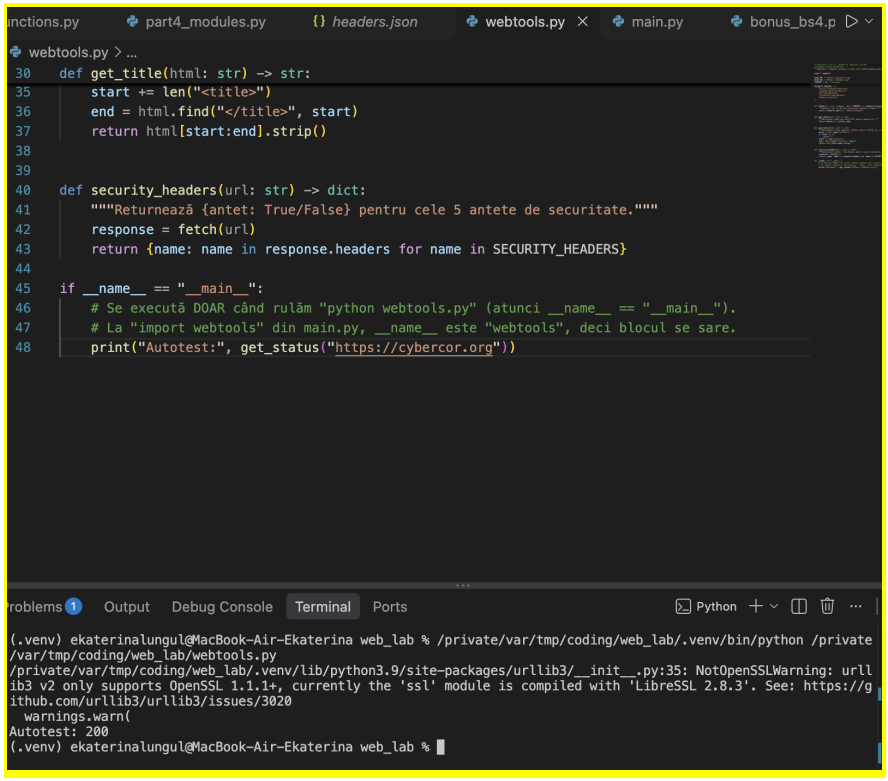


Fig.5.46.1 - Fisierul webtools.py

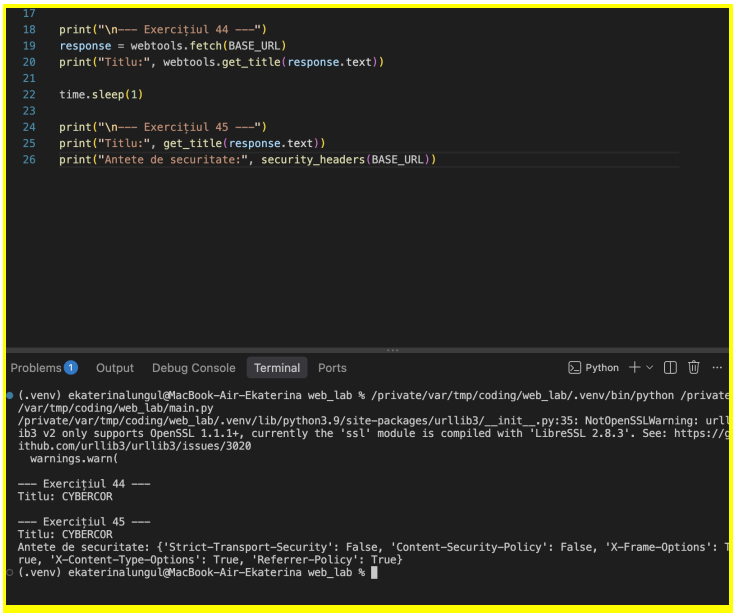


Fig.5.46.2- Fisierul main.py

Exercițiul 47. Constante de modul

Cerință: Adăugați în webtools.py constanta DEFAULT_HEADERS (User-Agent WebLab-<numele vostru>) și folosiți-o în fetch(); în main.py importați constanta și afișați-o.

[Am adaugat in webtools.py constanta DEFAULT_HEADERS={“User-Agent”: “WebLab-Boicu Ecaterina”} și am folosit-o in functia fetch(), prin parametrul headers.In main.py am important constanta cu from webtools import DEFAULT_HEADERS și am afisat-o în terminal.]

```
BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde


# Exercițiul 47: constantă de modul
DEFAULT_HEADERS = {"User-Agent": "WebLab-Boicu_Ecaterina"}
```

Fig.5.47.1 - Adaugarea constantei DEFAULT_HEADERS in webtools.py

```
print("\n--- Exercițiul 47 ---")
print(DEFAULT_HEADERS)
```

```
Problems 1  Output  Debug Console  Terminal  Ports                Python
venv/bin/activate
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab % /private/var/tmp/coding/web_lab/.
venv/bin/python /private/var/tmp/coding/web_lab/main.py
/private/var/tmp/coding/web_lab/.venv/lib/python3.9/site-packages/urllib3/__init__.py:35:
 NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module i
s compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(

--- Exercițiul 44 ---
Titlu: CYBERCOR

--- Exercițiul 45 ---
Titlu: CYBERCOR
Antete de securitate: {'Strict-Transport-Security': False, 'Content-Security-Policy': Fal
se, 'X-Frame-Options': True, 'X-Content-Type-Options': True, 'Referrer-Policy': True}

--- Exercițiul 47 ---
{'User-Agent': 'WebLab-Boicu_Ecaterina'}
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab %
```

Fig.5.47.2- Importarea constantei in fisierul main.py

Exercițiul 48. Argumente din linia de comandă

Cerință: Folosiți argparse în main.py, astfel încât programul să ruleze ca python main.py https://cybercor.org; transmiteți URL-ul primit funcțiilor în loc de BASE_URL.

[Am folosit modulul argparse in main.py , astfel incat programul sa ruleze ca python main.py https://cybercor.org. Adresa primită din linia de comanda este trimisă funcțiilor (fetch,get_title,security_headers) in loc de BASE_URL. La rulare programul a afișat URL-ul primit; Initial , la rulare python main.py fara adresa , programul se opreste cu mesajul error: the following arguments are required: url, aceasta eroare e normal. Ea apare cand programul porneste fara adresa site-ului. Cel mai probabil , eroare respectivă e din cauză ca VS Code porneste main.py fara agumente,de aceea , dupa ce am curățat terminalul , am scris adresa după numele fisierului : python main.py https://cybercor.org. La final , terminalul mi-a afisat rezultatul exercitiului 48 cu URL primit.]

main.py > ...

```python
# Laborator: funcții, metode și importuri pe web
# Student: Boicu Ecaterina

import argparse
import time

import webtools
from webtools import get_title, security_headers
from webtools import DEFAULT_HEADERS

# Exercițiul 45: dacă main.py ar avea și o funcție proprie get_title, ar exista
# două nume get_title în același spațiu de nume. Ultima definiție/atribuire o
# "suprascrie" pe cealaltă, fără nicio eroare. Conflictul se evită cu
# "import webtools" (webtools.get_title) sau cu un alias.

# Exercițiul 48: URL-ul vine din linia de comandă, nu din BASE_URL
parser = argparse.ArgumentParser(description="Raport despre un site web.")
parser.add_argument("url", help="adresa site-ului, de ex. https://cybercor.org")
args = parser.parse_args()
url = args.url

print("\n--- Exercițiul 44 ---")
response = webtools.fetch(url)
print("Titlu:", webtools.get_title(response.text))

time.sleep(1)

print("\n--- Exercițiul 45 ---")
print("Titlu:", get_title(response.text))
print("Antete de securitate:", security_headers(url))

print("\n--- Exercițiul 47 ---")
print(DEFAULT_HEADERS)

print("\n--- Exercițiul 48 ---")
print("URL primit din linia de comandă:", url)
```

Fig.5.48.1 - Primirea adresei site-ului in main.py din terminal in loc de BASE_URL

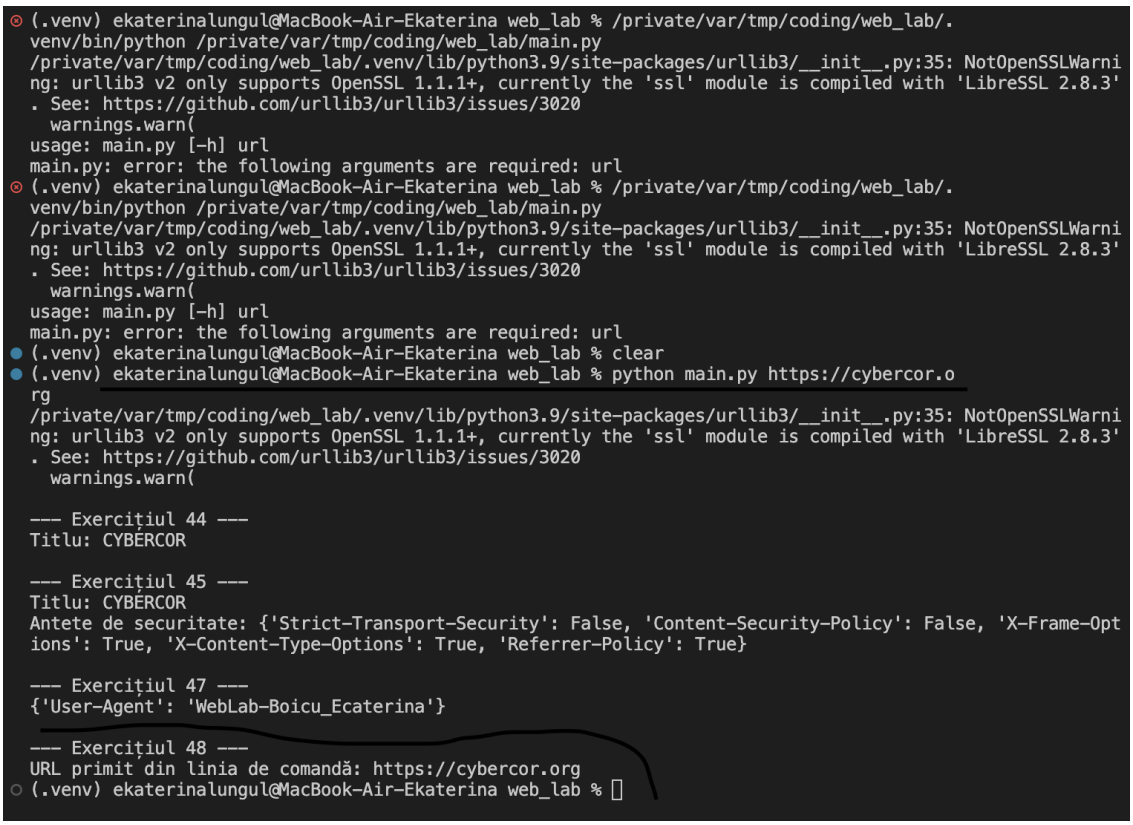


Fig.5.48.2 - Afișarea rezultatului terminalului (fără adresa inițial , iar apoi cu încadrarea adresei)

Exercițiul 49. Un raport CSV

Cerință: Cu modulul csv salvați rezultatul funcției check_paths() în report.csv, cu coloanele path, status și checked_at (ora: datetime.now().isoformat()).

[Am mutat functia check_paths() in webtools.py si am scris funcția save_csv(),care folosește modulul csv ca sa salveze rezultatul în raport.csv , cu coloanele path , status si checked_at(ora ,din datetime.now().isoformat()). In main.py am apelat check.paths pentru / ,/robots.txt si /sitemap.xml și am salvat rezultatul . Fișierul report.csv a fost creat în directorul web_lab.]

```
38  print("\n--- Exercițiul 49 ---")
39  time.sleep(1)
40  results = webtools.check_paths(url, ["/", "/robots.txt", "/sitemap.xml"])
41  webtools.save_csv(results, "report.csv")
42  print(results)
43  print("Salvat în report.csv")
```

Problems 1 | Output | Debug Console | Terminal | Ports | Python

```
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab % clear
python main.py https://cybercor.org
. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(

--- Exercițiul 44 ---
Titlu: CYBERCOR

--- Exercițiul 45 ---
Titlu: CYBERCOR
Antete de securitate: {'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-
ions': True, 'X-Content-Type-Options': True, 'Referrer-Policy': True}

--- Exercițiul 47 ---
{'User-Agent': 'WebLab-Boicu_Ecaterina'}

--- Exercițiul 48 ---
URL primit din linia de comandă: https://cybercor.org

--- Exercițiul 49 ---
{'/': 200, '/robots.txt': 200, '/sitemap.xml': 200}
Salvat în report.csv
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab %
```

Fig.5.49.1- Apariția dicționarului cu coduri în terminal

ons.py | part4_modules.py | webtools.py | main.py | report.csv | bonus_

report.csv

```
1  path,status,checked_at
2  /,200,2026-10-11T20:00:00.587129
3  /robots.txt,200,2026-10-11T20:00:00.587157
4  /sitemap.xml,200,2026-10-11T20:00:00.587165
5
```

Fig.5.49.2- Fișierul report.csv creat în directorul web_lab

Exercițiul 50. Proiect final: raportul site-ului

Cerință: Scrieți site_report(url) în webtools.py; funcția folosește funcțiile voastre pentru a colecta, a afișa și a salva în report.json: (1) codul de stare și URL-ul final, (2) titlul, (3) adresa IP, (4) lanțul de redirecționări pornind de la http://, (5) scorul antetelor de securitate, (6) zilele rămase până la expirarea certificatului, (7) numărul de legături interne și externe, (8) căile interzise din robots.txt.

[Am scris functia site_report(url) in webtools.py , care foloseste functiile mele pentru a colecta : codul de stare și URL-ul,titlul paginii ,adresa IP , lanțul de redirecționare pornind de la http:// ,scorul antetelor de securitate ,zilele rămase până la expirarea certificatului , numărul de legături interne și externe și căile interzise din robots.txt.Funcția afișează raportul in terminal si il salveaza in report.json.Am rulat-o din main.py cu comanda python main.py https://cybercor.org.]

```
--- Exercițiul 50 ---
=== Raport site: https://cybercor.org ===
Cod de stare: 200 | URL final: https://cybercor.org/
Titlu: CYBERCOR
Adresă IP: 104.21.91.71
Redirecționări: 301 -> https://cybercor.org/
Scor securitate: 3/5
Certificat: 52 de zile rămase
Legături: 1 interne, 0 externe
Căi interzise: niciuna
Salvat în report.json
(.venv) ekaterinalungul@MacBook-Air-Ekaterina web_lab %
```

Fig.5.50.1 - Rezultatul funcțiilor rulate in terminal


```
{} report.json > ...
{
  "url": "https://cybercor.org",
  "status_code": 200,
  "final_url": "https://cybercor.org/",
  "title": "CYBERCOR",
  "ip": "104.21.91.71",
  "redirects": [
    "301 -> https://cybercor.org/"
  ],
  "security_score": "3/5",
  "cert_days_left": 52,
  "internal_links": 1,
  "external_links": 0,
  "disallowed_paths": [],
  "generated_at": "2026-10-11T20:16:14.601326"
}
```


Fig.5.50.2- Fișierul report.json cu raportul salvat din terminal

Structura finală a directorului web_lab

Cerință: Directorul trebuie să conțină webtools.py, main.py, report.csv și report.json, iar comanda python main.py https://cybercor.org trebuie să afișeze raportul complet și să creeze report.json.

[Am verificat ca directorul web_lab contine webtools.py,main.py, report.csv și report.json. Comanda python main.py https://cybercor.org afiseaza raportul complet al site-ului și creeaza report.json.]

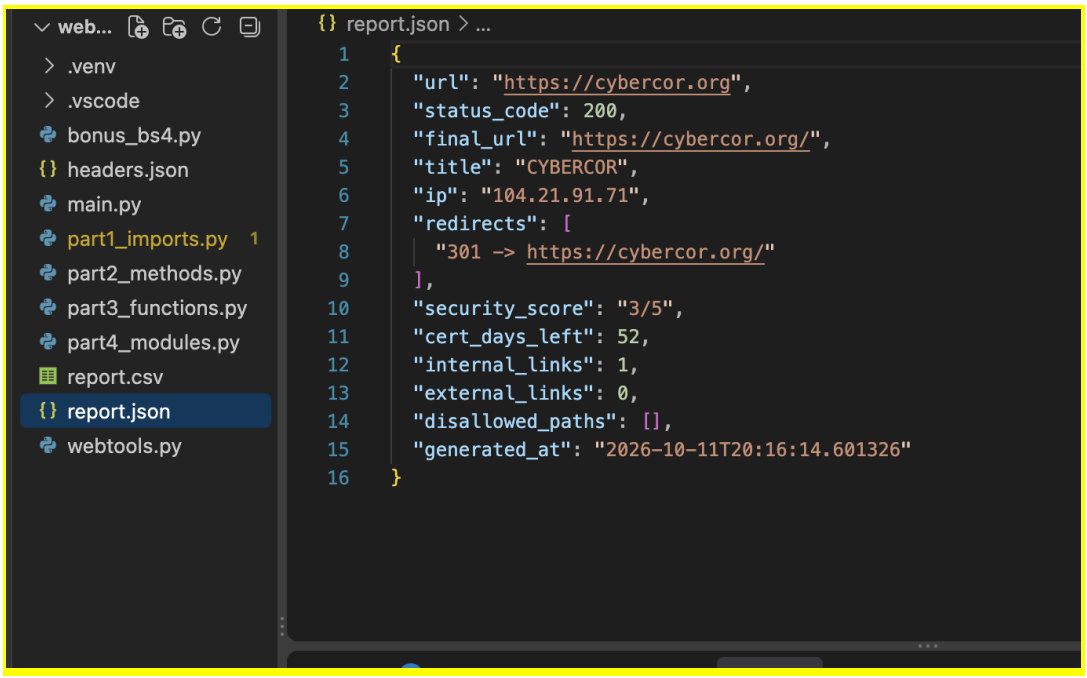
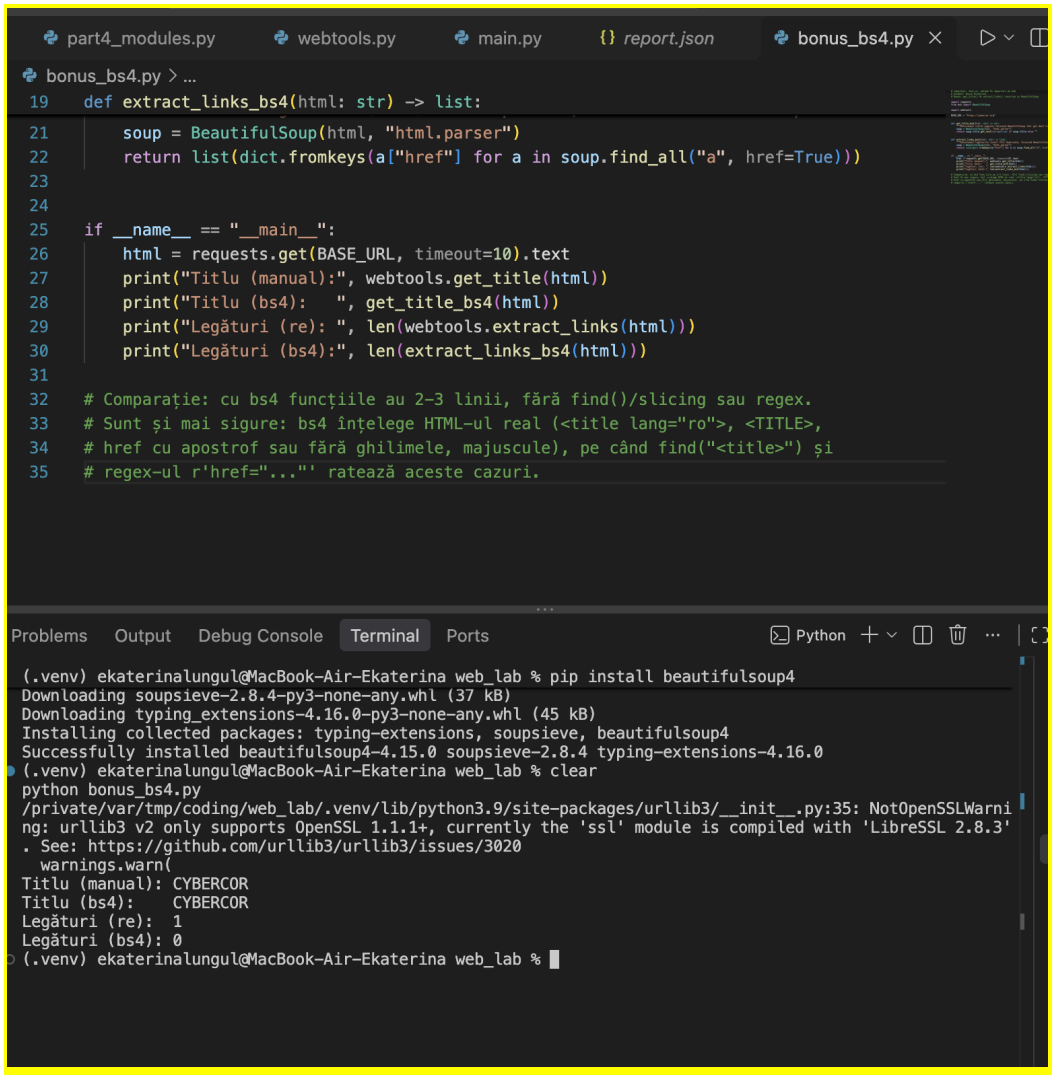


Fig.5.51 - Fișierele finale

Exercițiu bonus (opțional, până la 5 puncte)

Cerință: Instalați beautifulsoup4 și rescrieți get_title() și extract_links() folosind această bibliotecă; comparați cât de mult mai scurte și mai sigure devin funcțiile.

[Am instalat beautifulsoup4 și am rescris get_title() și extract_links() cu BeautifulSoup.funcțiile sunt mult mai scurte (2-3 linii, fără find(),felierea sau expresii regulate).Titlul obtinut este același (CYBERCOR) ,dar numărul de legături diferă:regex-ul a găsit 1 href ,pentru ca se potrivește cu orice tag (de exemplu <link>),în timp ce BeautifulSoup a găsit 0 ,deoarece pagina nu are tag-uri<a>.Astfel varianta cu BeautifulSoup este mai sigură și mai precisă , pentru ca înțelege structura HTML-ului,nu doar potriveste text.]



Screenshot nr. 57

CONCLUZII

In aceasta lucrare de laborator nr.5 la obiectul “Tehnici de Programare” ,am studiat importurile (import, from…import ,alias-uri ,module standard și externe),metodele si atributele obiectelor web (răspuns HTTP,antete ,șiruri de caractere ,cookie-uri , sesiuni) , scrierea funcțiilor proprii (parametrii impliciti ,docstring-uri ,adnotări de tip , tratarea erorilor) și utilizarea modulelor urllib.parse,re,html.parser,hashlib,json,socket si ssl.Am creat modulul propriu webtools.py si programul main.py, care primeste adresa site-ului din linia de comanda.Rezultatul final este funcția site_report(), care genereaza raportul unui site și îl salveaza in report.json, alaturi de report.csv.Am respectat regulile de lucru: timeout la fiecare cerere,pauze intre cereri si doar cereri de citire către site.